

Exact Filtered Vector Search Across Execution Environments: Why Scalar Routing Rules Fail and What a Router Must Measure

Akshat Singh Kushwaha
akshatsingh14372@outlook.com
<https://github.com/a3ro-dev/qenlo>

September 2026

Abstract

Filtered vector search chooses an execution route after a predicate restricts the candidates. We study that decision in Qenlo, an embedded vector store with exhaustive CPU, portable WGPU, and optional tensor execution. A scalar *EDB* rule selected with zero errors on 31 development CPU/GPU pairs fails a preregistered RTX 4050 suite, choosing the slower forced route in 8 of 16 workloads and reaching 235.7% maximum regret. Separate 384-dimensional, batch-one cohorts reverse CPU/WGPU order at different eligible counts (2K–3K versus 4K–6K). A partial single-host RTX 4090 campaign, with 227 of 260 planned block summaries and two batch-16 cells incomplete, adds three findings. First, same-cell repeat blocks differ by up to about $2\times$, well above the 25% regret gates used to judge routers; slow GPU blocks coincide with low end-of-run clock snapshots. Second, representation can matter more than backend. At 0.1–1% eligibility, compact GPU row lists are $3.2\text{--}6.1\times$ faster than shader predicates or dense masks, a margin that clears a conservative noise screen, whereas predicates are faster at full eligibility and batch one. Host row materialization takes 51% of a 30%-eligibility compact-row call. Third, on one host and binary, GPU wins at $EDB = 614,400$ (batch 16) while CPU wins at $EDB = 1,152,000$ (batch one) in 9 of 10 noise-control blocks. This post-hoc replication of the held-out ordering contradiction does not rest on one near-tie cell. The audit also found that the benchmark harness multiplied batch-level transfer counters by batch size; affected values are corrected. A post-hoc *EDB* threshold has 3.2% maximum regret on the held-out suite, so not every scalar policy is rejected. Routing is a joint backend, representation, and preparation decision whose certification needs block-replicated, noise-calibrated end-to-end tests. No router or threshold is validated here.

1 Introduction and research questions

A filtered query over 100,000 vectors may score only one hundred candidates. The remaining work can be dominated by finding those candidates, dispatching a device, selecting results, or returning them through an application interface. Conversely, an unfiltered batch may amortize dispatch enough to favor exhaustive GPU execution even for a modest collection. Collection size and GPU availability alone provide an incomplete description of the execution problem.

Qenlo supplies a concrete system in which to examine this boundary. Canonical records are shared by rebuildable search structures; changing routes should preserve the logical result contract. The archive contains several generations of implementation and experiment design. Earlier studies emphasize eligible cardinality, predicate representation, and routing regret. The September 5 campaign adds small collections, tensor adapters, resource diagnostics, lifecycle measurements, and a rejected selector. A later preregistered router suite supplies an actual held-out falsification. This paper recursively inventories retained sample series and reduces previously contextual Phase 0 and Phase 2 archives without treating historical or candidate results as measurements of the final source tree.

We ask four questions. Under what measured conditions does the preferred exhaustive route change? How much of a completed call comes from eligibility preparation, selection, synchronization, or derived-state preparation rather than scoring? How large is repeat-measurement variation relative to route differences, and therefore what can a routing gate certify? What deployment conclusions survive differences in numerical accuracy, APIs, persistence, runtime ownership, and evidence quality?

The contribution is a systems study, not a new asymptotic nearest-neighbor algorithm. It combines localized crossovers, an eligibility-materialization ablation, a failure-preserving small-collection campaign, heterogeneous-device observations, a sparse-only CPU optimization, and development/held-out routing evaluation. A partial block-replicated campaign then measures a same-cell noise floor and a same-host representation-by-batch ablation (Section 9). A claim ledger records which measurements support each conclusion. Rejected policies and optimizations retain scientific value, while their latency does not become a promise about the retained implementation.

The central argument has three steps. The held-out suite and cross-environment cohorts reject the scalar rules actually proposed. The block-replicated campaign shows why a better scalar is the wrong goal: representation and host preparation change completed-call latency by factors comparable to or larger than the CPU/GPU choice, and same-cell repeat variation is large enough to flip labels near parity. The deployment consequence is a list of quantities a router must choose and measure, not a fitted rule.

2 System model and semantic contracts

Let the live canonical collection at generation g contain normalized FP32 vectors $x_i \in \mathbb{R}^D$, public identifiers i , and metadata m_i . A predicate P defines

$$S_P(g) = \{i : i \text{ is live at } g \wedge P(m_i)\}, \quad E = |S_P(g)|. \quad (1)$$

For query q , exhaustive search evaluates eligible rows and returns up to k results ordered within each implementation by computed cosine distance and identifier. The ideal cosine distance is

$$d(q, x_i) = 1 - \frac{q^\top x_i}{\|q\|_2 \|x_i\|_2}. \quad (2)$$

A batch contains B queries. Scoring work scales with BED , but that expression omits predicate traversal, memory access, selection, and completed-call overhead.

Exhaustiveness and numerical agreement. We use “exact” in the algorithmic sense of exhaustive candidate coverage. Stored FP32 values, CPU accumulation, GPU arithmetic, and an FP64 reference need not yield identical rankings near ties. Oracle recall is reported independently. Historical dense native rows have recall@10 of 0.99998 despite exhaustive execution; corrected small-campaign rows have recall 1.0. In the E0/E2 campaign, all 20 summaries at 100% eligibility and batch one report exactly 0.99998 for every engine, CPU included, while the other 207 report 1; the dense value is a shared oracle-boundary property of this corpus, not an engine defect, and it is reported unrounded. Neither proves universal FP64 equivalence. ANN remains approximate even when observed recall is one. Deterministic engine ordering does not prove common FP64-oracle tie membership.

Canonical and derived state. The Rust store owns vectors, identifiers, metadata, tombstones, and generation. CPU scan layouts, WGPU residency, ANN indexes, and eligibility plans are derived state. Canonical vectors are FP32, normalized with an FP64 norm accumulator at ingestion and query boundaries. Restoration validates stored bytes without renormalizing them. Durable collections add snapshot and write-ahead-log behavior; in-memory benchmark collections have no durable generation. Source inspection supports these mechanisms, not universal power-loss durability or arbitrary concurrent or crash schedules.

3 Execution paths

CPU and portable GPU. Historical CPU artifacts describe FP64 accumulation with AVX2/FMA where available. Audited current CPU code also uses a bounded FP32 candidate stage with FP64 rescoring and reference fallback. Not every CPU arithmetic operation is FP64. These are Qenlo implementations, not bounds on optimized CPU performance. WGPU uses WGS� shaders and native graphics backends. WebGPU and WGS� specify a programming model, not performance portability [14, 15]; positive execution evidence is limited to named devices.

Resident GPU vectors use compact eligible rows, dense masks, or supported shader predicates. Bounded chunks emit candidates for host merging. Under a common total order, an item outside its chunk’s top k cannot enter the global top k , since at least k items in that chunk precede it. This standard reduction explains bounded readback; it does not remove floating-point differences.

Eligibility compilation. An `EligibilityPlan` binds a predicate result to generation and records eligible count, representation, row IDs, and preparation diagnostics. One-pass compilation avoids counting eligibility and traversing it again to materialize rows. A bounded cache keyed by generation and predicate reuses plans only while that identity remains valid. Exact row execution validates ordering, bounds, liveness, and predicate membership. The historical ablation holds GPU scoring fixed across two-pass, one-pass, and warmed cached paths; it does not recalibrate the earlier laptop’s crossover. After the E0/E2 audit, required shader-predicate batches count eligible rows directly from canonical metadata indexes without allocating or sorting a host row list; automatic execution still retains rows because a CPU route or device-failure fallback may consume them. This behavior-preserving source change is not reflected in campaign latency and is not a representation-routing policy.

Tensor execution. The optional Python `TorchIndex` owns vector and ID tensors and supports CPU, CUDA, and MPS selection in source; only CPU and CUDA have measurements in this paper. The repository’s conformance test is CPU-focused, so source-level device support should not be read as positive evidence for every CUDA or MPS device. A collection-derived snapshot records its generation and rejects staleness at search entry during sequential use. It neither holds a native collection lock throughout computation nor rechecks afterward; concurrent mutation guarantees are unestablished. A detached array index has no durable collection identity. Both are derived computation, not another durable database. Replays prepare filtered matrices before query timing. Tensor allocation fields exclude allocator and library overhead.

Mutation and route observability. Resident updates append rows or update liveness without rebuilding when capacity and layout permit; rebuild remains the fallback. CPU-only operation avoids GPU initialization. Required-GPU mode returns errors on failure, while automatic mode records fallback. Static routing and hardware-bound profiles exist, but no retained experiment validates a calibrated profile on held-out decisions. File-backed commits validate and stage durability before publication; this statement does not apply to in-memory collections.

4 Methodology and evidence boundaries

A comparison needs source and archive identity, hardware and software, dataset and query partition, $N, D, B, k, E/N$, predicate representation and semantics, API, timing, and correctness gate. Numerical observations stay within named cohorts; cross-cohort conclusions are qualitative synthesis. Historical endpoints with differing revisions remain descriptive context, not controlled ablations.

Timing. Native timing starts at the host search call and ends with host-visible ordered results, including in-call eligibility, transfers, dispatch, synchronization, readback, and merge. Load, construction, warmup, and oracle work are excluded. External small-campaign adapters also return host-visible identifiers and distances but receive a prefiltered canonical matrix. Their build field includes matrix and index construction; the metadata traversal creating eligible IDs precedes that timer. Even `build_and_filter_prepare_ns` omits part of preparation. Chroma includes Python collection bindings and serialization. These are scoped completed calls, not identical database APIs.

CUDA events are separate from host calls. Scoring, selection, and backend diagnostics overlap: summing them or subtracting them from P95 invents an unmeasured phase. Lifecycle records separate mutation, collection opening, and first search. Throughput fields including validation in their wall interval are not inverse query latency and are not used for cross-engine QPS rankings.

Samples and correctness. New synthetic cells generally use three repetitions of 64 evaluation queries: 192 batch-one calls, 24 batch-eight calls, or three batch-64 calls. The real fixture uses 5,000 queries per repetition and 939 batch-16 calls across three runs. Matrix `sample_count` means native batch calls but external replay repetitions; Table 3 restores common calls and runs notation. Per-run nearest-rank percentiles are reduced by the lower median. Replays preserve native query order and truth and validate membership, distances, eligibility, and recall outside timing. Repeated queries are not independent deployment workloads.

Historical Windows and eligibility cells use five runs of 5,000 held-out calls, $k = 10$, $B = 1$, and 200 warmups; the statistic is median run P95. A6000 tables summarize per-call distributions within cells; synthetic A6000 has 500 calls per system and cell. Android and Arc provide aggregate percentiles. No figure claims significance or

draws error bars. Lines are guides. Sequential execution, post-hoc choices, and small run counts limit causal and statistical interpretation.

E0/E2 uses blocks as the resampling unit. Each block is one fresh process per engine, in randomized engine order, with a recorded 30-second cooldown, 200 warmups, and three repetitions of 5,000 held-out queries: 15,000 batch-one calls or 939 batch-16 calls per block. A block’s statistic is the lower median of its three run percentiles. Cell medians and 95% intervals come from 20,000 percentile-bootstrap draws over blocks; paired engine ratios bootstrap the median per-block log ratio. Repetitions inside a block are never treated as independent. Intervals describe repeat-block variation for one workload on one host, not a population.

The archive-wide census hashes canonical-LF CSV bytes. It includes tracked and selected archive-internal `samples.csv`, `raw_samples.csv`, and lifecycle records; exact mirrors contribute one unique series. The twelve schemas have different sample units. Their 1,826,130 rows are never pooled into a confidence interval or treated as independent query draws. Numerical synthesis below uses only source-compatible comparisons, while the complete disposition ledger remains machine readable.

5 Evidence cohorts and source identity

Table 1: Cohort map. Observations describe archive volume, not independent replications. Codes identify source roles below.

Cohort	Hardware / data	Scope	Evidence
Small S0/S1/S2	Ada, A40, 4090; generated, AG News	1K–100K, 128–768D	182 status rows
Localization H1	RTX 4050/DX12; AG News	100K, 384D, $E = 2K$ –10K	300,000 raw calls
Endpoints H0	RTX 4050; AG News	100K, sparse/dense	225,000 raw calls
Eligibility H2	RTX 4090/Vulkan; AG News	100K, 384D, three modes	300,000 raw calls
Linux H3	Cloud CPU; AG News	100K, 384D, seven engines	10,500 raw calls
A6000 real H4	CUDA; TopK ModernBERT	1M, 768D, strict predicate	20,000 raw calls
A6000 synthetic H5	CUDA; generated FP32	1M, 768D, four E	4,000 raw calls
Android H6	Mali-G615/Vulkan	10K/100K, 384D	21 aggregate cells
Intel Arc H7	Arc/Vulkan	10K/100K, 384D	21 aggregate cells
Router R1	RTX 4050/DX12; generated	16 held-out shapes	48 raw route series
Phase 0 P0	RTX 4090/Vulkan; AG News	100K, 384D, $E = 1K$ –100K	19 cells
Phase 2 P2	A40 CPU; AG News	100K, 384D, three E	8 reduced rows
Noise/repr. E0/E2	RTX 4090/Vulkan; AG News	100K, 384D, $B \in \{1, 16\}$, $E = 100$ –100K	227 of 260 block summaries

S0 is the frozen archive `f37e744c`, S1 is the experimental archive `730c0500`, and S2 is the corrected supplement `bb195fed`. Both S1 and S2 contain the lane-minimum candidate. The local WGSF is byte-identical to S0’s simpler selector. The manifests describe a dirty worktree around HEAD `b22d3b5`; the archive hashes identify the measured source more precisely than that commit alone. The matrix label `current-gpu` is an artifact key, not a claim about final local behavior.

H1 records common revision `3e2a4a9` in a dirty worktree; its configurations and outputs are part of the evidence identity. H0 contains several revisions. H2 is archive `26e0ccc0`, and H3 is `bb51987`, mirrored and counted once. H4 and H5 use different A6000 environments and research CUDA implementations. H6 and H7 identify device-lab run IDs rather than reconstructible executable archives. R1 is SHA-256 `8e3b5fe6`; P0 is `7292fd1c`; and P2 is `ac3afda4`. E0/E2 is the partial archive `e4ca1336`; its manifest attests source `e5d85ad`, benchmark binary `b28e16ff`, and dataset `ba87a322`, but the binary and dataset are not in the archive, so only the runner hash and the recorded dataset CRC32 `beb3c848` are independently checkable. Its configurations report `git_revision=unavailable`; the source identity is an attestation. Executable revisions and hosts differ across cohorts, so cross-cohort synthesis uses winner contradictions rather than pooled latency.

Windows data derive from DTU AG News embeddings [9]; Linux uses a separately prepared fixture. Equal dataset names do not imply equal bytes or partitions. The small campaign retains its own fixture and checksums.

The A6000 cohort uses TopK Bench’s million-vector, 768-dimensional ModernBERT corpus [13]; its strict and provider-compatible predicates are separate. Appendix B expands the provenance and exclusions.

6 Small-collection campaign

6.1 Coverage and common sweep

The matrix has 182 rows: 131 completed, 42 marked **failed_or_unavailable**, seven failed, and two marked invalid-harness. Only 130 rows qualify. The completed USearch batch-64 row remains visible at recall 0.3453125. The manifest records fifteen configurations: ten primary offers and five reference, deep, retry, or fix follow-ups. These are neither fifteen device replications nor ten successful devices. Results came from RTX 2000 Ada, two A40 configurations, and a reference RTX 4090; unavailable L4, RTX 3090, A6000, and community offers remain visible.

Table 2: Common synthetic workloads, $k = 10$. Six selected cells, not a factorial sweep.

Cell	N	D	B	E/N
C1	1,000	128	1	100%
C2	1,000	384	16	1%
C3	10,000	128	16	100%
C4	10,000	384	1	1%
C5	100,000	128	1	1%
C6	100,000	384	16	100%

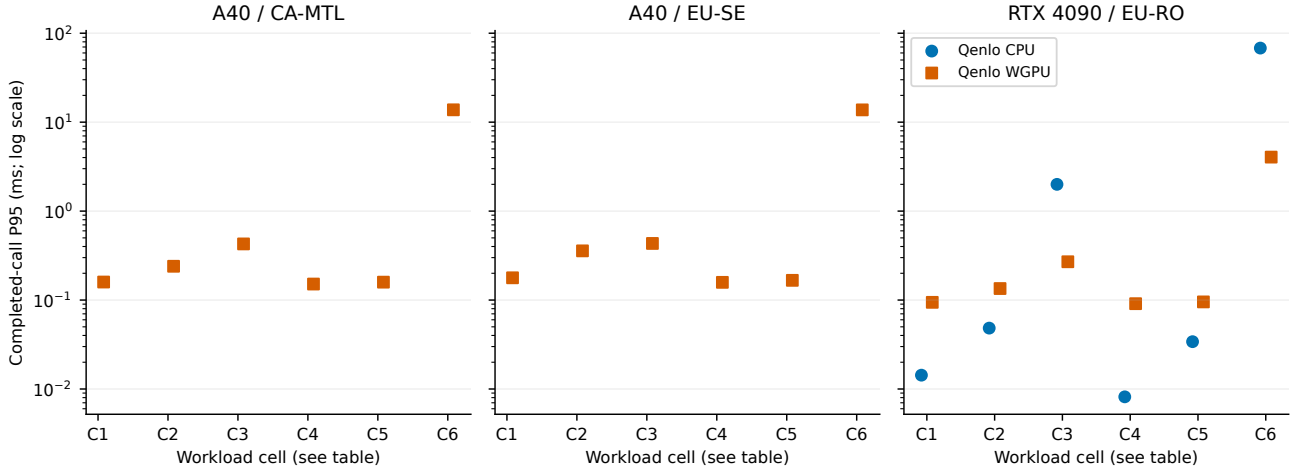


Figure 1: S1 730c0500, synthetic common cells, native completed-call P95. Panels are separate hosts. WGPU uses the candidate; no interpolation/error bars. Current-CPU rows are absent on A40, so those panels do not establish CPU/GPU pairs. All displayed FP64 recalls are 1.0. Vertical axis is logarithmic.

On the reference RTX 4090, CPU wins C1, C2, C4, and C5, while WGPU wins the dense batched cells C3 and C6. In C5, 100K total rows leave 1K eligible: CPU takes 0.0341 ms P95 versus WGPU’s 0.0953 ms. In C6, WGPU takes 4.0479 ms versus CPU’s 68.0575 ms. Dimension, batching, and eligibility change together, so these are route observations rather than isolated size effects. The A40 configurations should not be combined into a pooled device curve.

6.2 Corrected 100K-by-768 supplement

S2 ran on Linux with an RTX 4090 and Vulkan, driver 580.159.04, and an AMD Ryzen Threadripper 7960X. The generated data record has CRC32 50b69280. Table 3 verifies all twelve rows against the raw records and matrix. These rows used the candidate subsequently rejected by the local source.

Table 3: S2, 100K $\times$ 768, completed-call ms. FP64 recall 1.0 throughout. Calls/runs counts batch calls across runs; replays follow native batches. External matrices are prefiltered outside timing.

Workload	Engine	P50	P95	Recall	Calls/runs
Full, B=1, k=1	Qenlo CPU	9.010	9.409	1.0	192 / 3
Full, B=1, k=1	Qenlo WGPU	0.870	0.897	1.0	192 / 3
Full, B=1, k=1	FAISS Flat	14.483	14.800	1.0	192 / 3
Full, B=1, k=1	NumPy	14.507	15.343	1.0	192 / 3
Full, B=1, k=1	Torch CPU	18.324	18.695	1.0	192 / 3
Full, B=1, k=1	Torch CUDA	0.654	0.671	1.0	192 / 3
10%, B=8, k=64	Qenlo CPU	27.402	28.599	1.0	24 / 3
10%, B=8, k=64	Qenlo WGPU	0.842	0.896	1.0	24 / 3
10%, B=8, k=64	FAISS Flat	2.716	3.055	1.0	24 / 3
10%, B=8, k=64	NumPy	5.972	6.202	1.0	24 / 3
10%, B=8, k=64	Torch CPU	2.847	2.936	1.0	24 / 3
10%, B=8, k=64	Torch CUDA	0.370	0.384	1.0	24 / 3

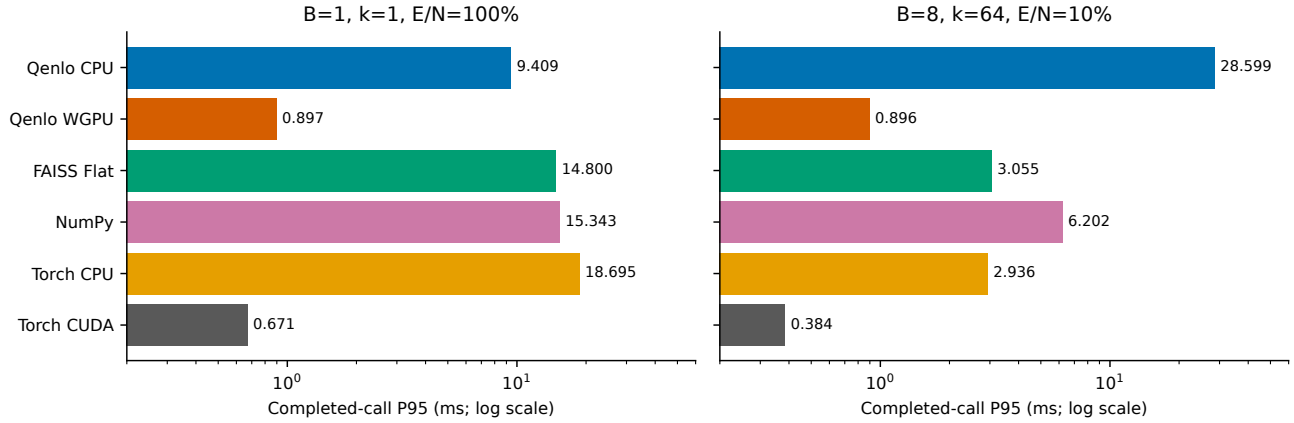


Figure 2: S2 **bb195fed**, RTX 4090/Vulkan, generated 100K $\times$ 768. Host-call P95, logarithmic horizontal axes, no error bars. Eligibility/batching differ between panels. WGPU is the rejected candidate; all FP64 recalls 1.0. FAISS Flat, NumPy, Torch use prefiltered matrices.

For the unfiltered batch-one workload, WGPU is 10.49 times faster than native CPU at P95, while Torch CUDA is 1.34 times faster than WGPU. At 10% eligibility and batch eight, the corresponding ratios are 31.91 and 2.33. These are archived calls. Fresh matched runs are required before attaching the WGPU values to the final full-rescan selector. Tensor latency excludes deployment and persistence costs around a matrix.

6.3 Real embeddings

The S1 real AG News workload has $N = 100K$, $D = 384$, $B = 16$, $k = 10$, and $E/N = 1\%$. WGPU P95 is 0.171958 ms, CPU is 0.941633 ms, FAISS Flat is 0.242735 ms, NumPy is 0.749214 ms, Torch CPU is 0.268323 ms, and Torch CUDA is 0.351387 ms; all have FP64 recall 1.0. WGPU leads Torch CUDA in this workload, reversing their order in distinct S2 synthetic cells. The change across workload and source conditions does not isolate its cause. One fixture cannot establish robustness to different models, geometries, or correlated metadata. ANN rows and their recall remain in Appendix A.

7 Historical crossover and eligibility preparation

7.1 A localized exhaustive winner reversal

Matched H1 searches use 100K $\times$ 384 AG News on an RTX 4050 with DX12, $B = 1$, and $k = 10$. CPU takes 0.5198 ms P95 versus compact-row WGPU’s 0.6340 ms at $E = 2K$. At 3K, WGPU takes 0.8144 ms versus CPU’s

0.9563 ms and remains faster through 10K. The reversal is bracketed by (2K, 3K); it is not a fitted threshold. All localization recalls are 1.0. Compact-row timing includes the duplicated count and materialization traversal.

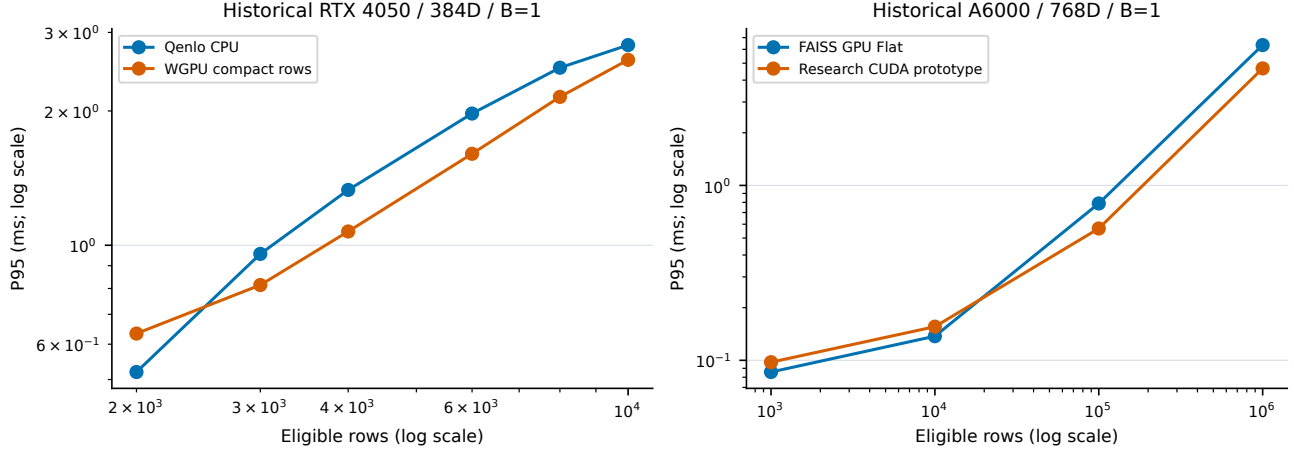


Figure 3: Separate historical panels. Left: H1 3e2a4a9, RTX 4050/DX12, AG News 100K×384, $B = 1, k = 10$, compact rows, median five-run P95, FP64 recall 1.0. Older endpoint revisions omitted. Right: H5 A6000 research CUDA/FAISS, generated 1M×768, prefiltered rows, $B = 1, k = 10$, P95 over 500 calls/system/cell; unordered top-10 agreement without independent FP64 oracle. Logarithmic axes; lines guide the eye, not fitted laws.

The shipped rule chooses CPU for $E < 4,096$ and batches below eight. Counterfactually applied to six H1 cells, it misroutes 3K and 4K, with relative P95 regret of 17.4% and 23.9%. For cell c and valid measured routes $\mathcal{R}_c$,

$$R(c) = \frac{T_{\pi(c),95}}{\min_{r \in \mathcal{R}_c} T_{r,95}} - 1. \quad (3)$$

These compare separate run-level percentiles, not paired queries. Equal weighting of six post-hoc cells is not an application distribution. A profile mechanism in code does not prove adaptive routing reduces regret.

Counterfactual regret of the shipped 4,096-row rule, RTX 4050

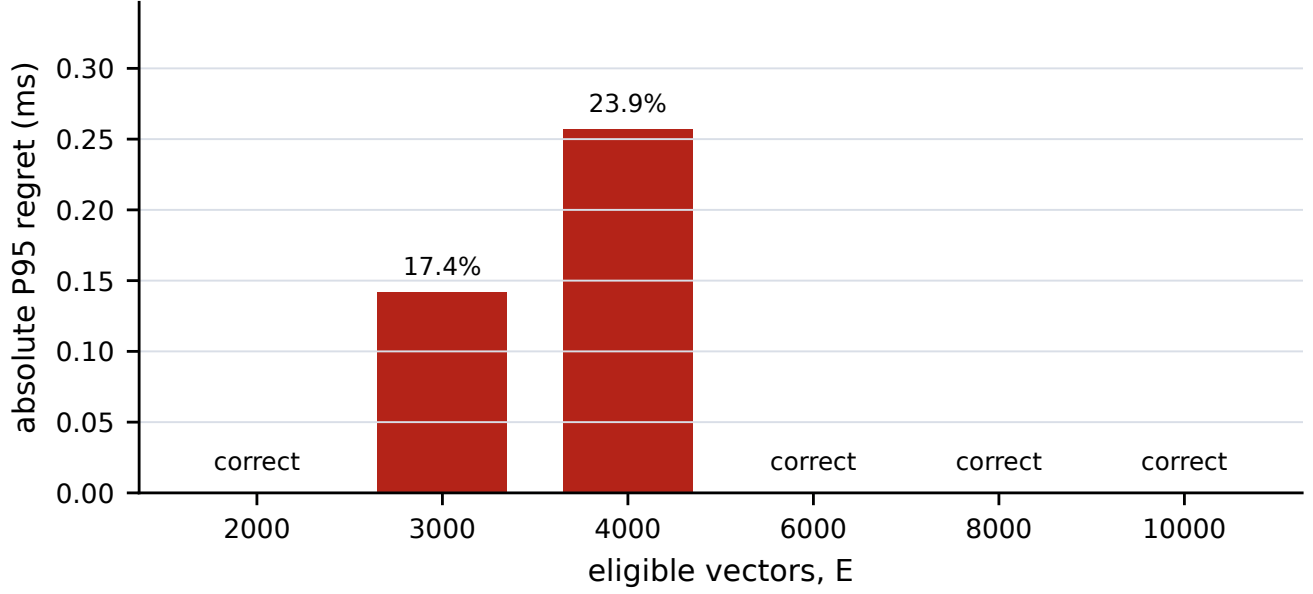


Figure 4: H1 RTX 4050, same 100K×384 compact-row cells and FP64 recall 1.0 as Figure 3. Completed-call P95 regret uses median five-run results and frozen 4,096-row rule. Annotations give relative regret. Descriptive counterfactual; no routing traces/confidence bars.

7.2 One-pass and cached eligibility

H2 uses a separate RTX 4090 with Vulkan, a Ryzen 9 7950X, and driver 570.211.01. Its 100K×384 AG News ablation holds scoring and queries fixed across five runs per mode at $E \in \{1K, 4K, 6K, 100K\}$. One-pass preparation lowers P95 from 0.131736 to 0.117870 ms at 1K, from 0.305281 to 0.208009 ms at 4K, and from 0.401130 to 0.271618 ms at 6K, reductions of 10.5–32.3%. The warmed cache lowers the sparse values by 24.3–57.3%. The dense improvement is smaller: 1.786170 to 1.720476 ms for one-pass preparation, or 1.596024 ms with the cache.

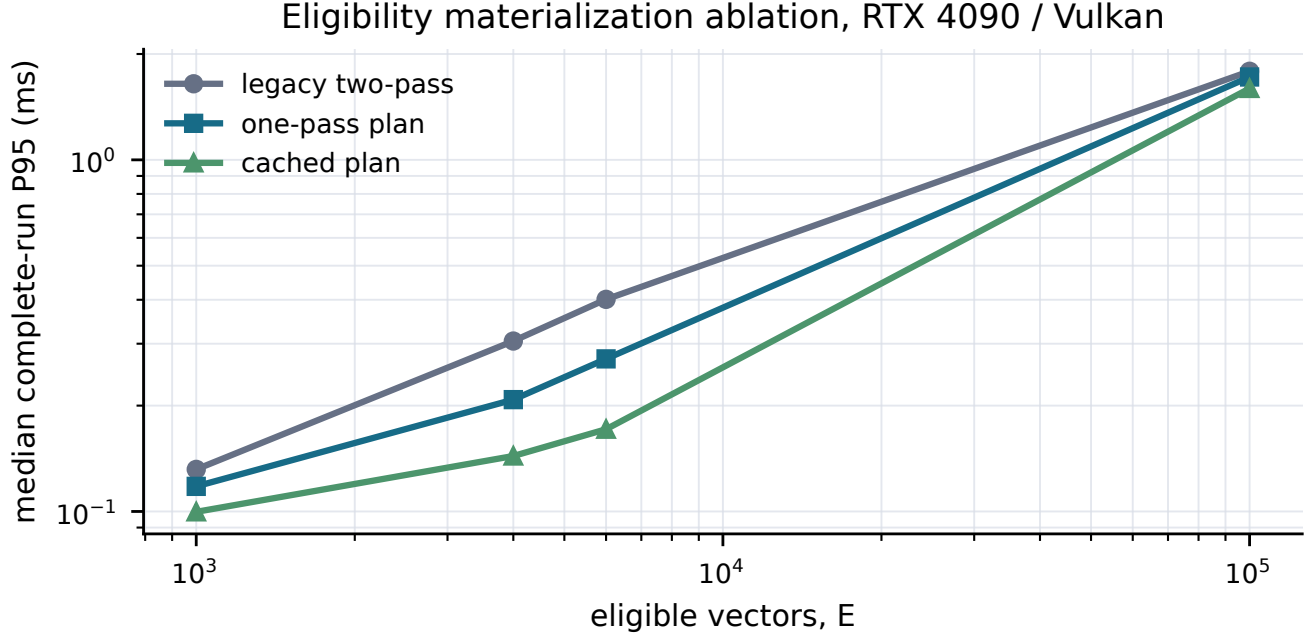


Figure 5: H2 26e0ccc0, RTX 4090/Vulkan, 100K×384 AG News, $B = 1$, $k = 10$. Median five-run completed-call P95, 25,000 calls/point. Sparse FP64 recall 1.0; dense 0.99998. Sequential modes, logarithmic axes, guide lines, no uncertainty intervals.

The cache benefit requires an identical predicate and generation; a mutation invalidates that premise. Because H2 has no matched CPU cells and uses another host, it cannot relocate H1’s crossover. It does establish that duplicated eligibility work affects a completed call.

8 Archive-wide routing falsification

8.1 Development fit and preregistered failure

The EDB candidate routes to WGPU when $EDB \geq 10^6$. It was selected from 31 compatible development pairs and classified every development winner. R1 then held hardware and source to its archived RTX 4050/DX12 suite and varied three query shapes, $k \in \{1, 10, 64\}$, and work on both sides of the frozen threshold. All forced CPU, forced compact-row WGPU, and automatic cells pass their recall and filter gates. The frozen scalar chooses the wrong forced-route winner in 8 of 16 workloads. Median choice regret is only 0.4% because half the cells have zero regret, but the maximum is 235.7%; the preregistered 25% limit rejects the policy.

Table 4: Development and held-out evaluation of the frozen $EDB = 10^6$ rule. Regret compares separate forced-route median-run P95 values; it does not substitute automatic-route timing.

Split	Cells	Wrong	Median %	Max %	Decision
Development	31	0	0.0	0.0	threshold selected
Held out	16	8	0.4	235.7	rejected

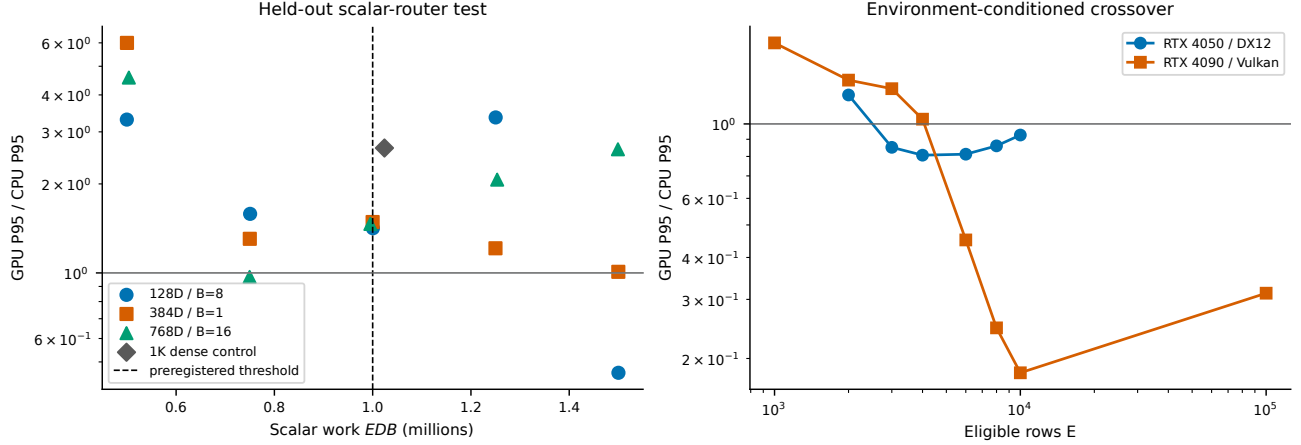


Figure 6: Left: R1, sixteen preregistered RTX 4050/DX12 workloads. The dashed line is the frozen $EDB = 10^6$ threshold; ratios below one favor WGPU. Near-equal scalar work produces different winners. Right: separate $100K \times 384$, $B = 1$, $k = 10$ H1/P0 cohorts; the P0 source, host, and runtime differ. Lines guide the eye, latencies are not pooled, and no error bars are claimed.

Near-equal work with different winners alone does not disprove a threshold: a boundary could lie between those values. A genuine ordering contradiction occurs at $k = 10$: $768D/B = 16$, $E = 61$ favors WGPU at 749,568 units (0.918 versus 0.947 ms), whereas $384D/B = 1$, $E = 1,953$ favors CPU at the larger 749,952 units (0.951 versus 1.2404 ms). Thus no increasing EDB threshold perfectly classifies these observed medians. All same- k ordering witnesses depend on the former, only 3.16%-margin cell; sequential run variability prevents a robust causal conclusion about factorization. A post-hoc threshold of 1,499,904.5 units leaves one error and 3.2% maximum regret. This is below the original regret limits but fitted to the test set, not new validation. The frozen policy failed; not every scalar policy has been rejected at practical regret limits.

The realized automatic path is also not interchangeable with the forced route selected by a policy. Across R1, automatic P95 ranges from 0.412 to 5.393 times the faster forced-route P95. This spread cannot be labeled routing overhead: sequential run drift, execution state, and path differences remain competing explanations. One path difference is concrete. R1’s raw samples show that the automatic arm’s 8,400 WGPU calls used shader-predicate eligibility, whereas the forced WGPU arm used compact rows, so auto/best mixes representation with backend. A router must therefore be evaluated end to end, not only by applying labels to forced-route percentiles.

Noise sensitivity of the held-out counts. A read-only audit of the R1 archive compares each automatic cell with the forced arm that ran the same backend. That A/A ratio lies outside $\pm 25\%$ at P95 in 9 of 16 cells, and P50 and P95 disagree on the winner in two cells, including the $768D$, $B = 16$, $E = 61$ ordering witness. The E0 campaign independently finds same-cell block ratios near $2\times$ on another host (Section 9). Consequently the count of 8 wrong choices is not noise-robust: several wrong labels sit near parity. The rejection itself survives, because its maximum regret of 235.7% (a $3.36\times$ ratio) exceeds every A/A ratio reported here, and because the ordering contradiction is replicated with block replication in Table 8. A 25% regret gate below the measured A/A band cannot certify a router in either direction.

8.2 Environment-conditioned crossover and CPU route specialization

P0 contributes eight CPU/compact-row WGPU pairs on a separate RTX 4090/Linux/Vulkan environment. Its observed reversal lies between 4K and 6K eligible rows, not H1’s 2K–3K bracket. At $E = 3K$ and 4K the two cohorts have opposite winners. Consequently no monotone E -only rule can classify all fourteen observed cells. Even the best post-hoc minimax threshold, just above 3K, retains 17.4% maximum regret; the 4,096 rule retains 23.9%. This is a falsification over these cells, not attribution to GPU model alone: source, host CPU, OS, API, driver, and execution order also differ.

P2 supplies another conditional route inside the CPU engine. On an A40 host, a certified FP32 candidate stage with FP64 reranking reduces completed-call P95 by 17.0% at $E = 1K$ and 28.0% at 4K, but increases it by 10.0% at 100K. The measured implementation therefore routes the optimization only through 4,096 eligible rows.

Dense single-thread FAISS CPU Flat takes 11.154 ms versus the frozen Qenlo baseline’s 15.250 ms, 26.9% lower latency, while recall is 0.99984 versus 0.99998. This is a scoped library-call comparison, not numerical equivalence or a database ranking.

Table 5: P2 A40 selective CPU candidate, 100K×384, $B = 1, k = 10$, five runs and 5,000 calls/run. Change is optimized/baseline P95 minus one.

E	Baseline ms	Optimized ms	Change %	Recall@10
1,000	0.1715	0.1424	-17.0	1.00000
4,000	0.6120	0.4404	-28.0	1.00000
100,000	15.2497	16.7750	+10.0	0.99998

9 Noise floor and representation ablation

9.1 Design, coverage, and provenance exceptions

The E0/E2 campaign ran on one Linux RTX 4090 with Vulkan, using 100K×384 AG News vectors, independently generated synthetic metadata, a shared timestamp-range predicate (all rows at 100%), $k = 10$, one-pass row preparation, and the recall gate 0.99. E0 repeats one cell, $B = 1$ and $E = 3,000$, for CPU and compact-row WGPU in ten blocks. E2 crosses four engines (CPU, WGPU compact rows, shader predicate, dense mask) with $B \in \{1, 16\}$ and $E \in \{100, 1K, 3K, 10K, 30K, 100K\}$, five blocks each. The eligible fraction is $f = E/100,000$ on one corpus; fixed E with varying N is untested.

The plan has 260 block summaries; 227 exist: E0 20/20 and E2 207/240. There is no completion marker. All missing slots are in the last two batch-16 cells: at $E = 30K$, CPU has one of five blocks and each GPU engine two of five (13 missing); at $E = 100K$ all 20 are missing (Appendix Table 18). Every present summary reports completion, a passed recall target, and zero filter violations. Two provenance exceptions remain visible. First, the gpu-mask process for $B = 1$, $E = 100$, block 2 wrote a complete summary and all three run rows but exited with code 101; the runner halted on it, and a runner resumed about 20 minutes later skipped that destination because its summary existed. The block is included, flagged, and changes the rows/mask P95 ratio only from 0.197 to 0.198 when omitted. Second, the CPU destination for $B = 16$, $E = 30K$, block 1 contains configuration, metadata, truth, and tuning files but no runs or summary; it was in progress when the campaign stopped 6.45 hours after the first run and is counted as missing, not as a failure or a latency. The runner log also contains 150,941 repeated errors for a missing continuation-script path and the stop script logged a shell syntax error; these are control-plane anomalies, not evidence against completed summaries. The retained tarball has 6,575 trailing non-gzip bytes that stop GNU tar; Python’s `tarfile` reads all 2,132 members, and the archive hash matches the supplied value.

9.2 E0: repeatability of one cell

Table 6: E0, RTX 4090/Vulkan, $B = 1$, $E = 3,000$, ten blocks per engine, milliseconds. The interval is a block bootstrap of the median. Spread is the largest max/min ratio among a block’s three runs. The all-pairs column is the 95th percentile of $|\ln(L_j/L_i)|$ over 45 dependent block pairs; 0.66–0.69 corresponds to 1.94–1.99×. The last column counts pairs beyond a 25% ratio among five disjoint adjacent pairs and 45 dependent pairs. No block is removed.

Engine	Metric	Blocks	Median [95% CI]	Max spread	All-pairs P95 ln	> 25%
CPU	P50	10	0.369 [0.360, 0.386]	1.101	0.104	0/5; 0/45
CPU	P95	10	0.382 [0.375, 0.402]	1.550	0.688	1/5; 9/45
gpu-rows	P50	10	0.702 [0.557, 0.714]	1.208	0.661	2/5; 16/45
gpu-rows	P95	10	0.750 [0.614, 0.776]	1.312	0.592	2/5; 16/45

Repeating an identical cell does not reproduce latency within the 25% margins used by earlier routing gates. CPU P50 is stable, but one CPU block has a 0.756 ms P95, and compact-row WGPU is bimodal: eight blocks lie near 0.72–0.83 ms P95, while blocks 5 and 6 take 0.506 and 0.424 ms. A robust modified- z rule flags these

blocks; all remain in every statistic. Among 16 of 45 GPU block pairs, the ratio exceeds 25%. Across all $B=1$ GPU cells, the block with the higher end-of-run SM clock snapshot has the lower P95 in 80 within-cell pairs and the higher P95 in 31, with 114 ties; the E0 fast blocks end at 840 and 900 MHz, while slow blocks end at 405–480 MHz. Snapshots bracket each process rather than tracing it, so dynamic frequency scaling is a hypothesis for the bimodality, not an established cause. E0 measured one sparse batch-one cell, so its band may not transfer to other cells.

9.3 E2: representation and batch

Table 7: E2 paired ratios (left/right, median of per-block log ratios, 95% block bootstrap). Below one favors the left engine. n is paired blocks. †: the whole interval lies beyond the larger E0 all-pairs band for that metric (0.661 at P50, 0.688 at P95 in $|\ln|$), a conservative screen rather than a significance or equivalence test. $f = 1$ at $B = 16$ is missing; $f = 0.3$ at $B = 16$ has two GPU blocks and one CPU block. Rows/mask ratios are in Appendix Table 21.

B	f	n	Rows/predicate P50	Rows/predicate P95	Rows/CPU P50	Rows/CPU P95
1	0.001	5	0.186 [0.181, 0.189]†	0.198 [0.187, 0.201]†	10.851 [10.201, 11.083]†	10.665 [10.201, 11.465]†
1	0.01	5	0.276 [0.268, 0.333]†	0.315 [0.306, 0.331]†	2.587 [2.510, 3.162]†	2.824 [2.148, 3.187]†
1	0.03	5	0.396 [0.368, 0.643]	0.420 [0.368, 0.633]	1.209 [1.020, 2.004]	1.211 [0.834, 2.004]
1	0.1	5	0.862 [0.650, 0.925]	0.910 [0.578, 0.972]	0.347 [0.257, 0.377]†	0.377 [0.231, 0.405]†
1	0.3	5	1.002 [0.810, 1.043]	1.040 [0.963, 1.127]	0.449 [0.355, 0.466]†	0.447 [0.423, 0.581]
1	1	5	1.663 [1.315, 1.675]	1.673 [1.563, 1.796]	0.221 [0.175, 0.223]†	0.222 [0.210, 0.246]†
16	0.001	5	0.163 [0.145, 0.167]†	0.181 [0.148, 0.201]†	0.709 [0.705, 0.725]	0.661 [0.554, 0.729]
16	0.01	5	0.173 [0.171, 0.181]†	0.178 [0.169, 0.211]†	0.144 [0.138, 0.148]†	0.147 [0.120, 0.161]†
16	0.03	5	0.601 [0.494, 0.751]	0.666 [0.579, 0.835]	0.173 [0.146, 0.183]†	0.181 [0.164, 0.188]†
16	0.1	5	0.944 [0.916, 1.064]	0.961 [0.947, 1.551]	0.034 [0.033, 0.039]†	0.033 [0.026, 0.049]†
16	0.3	2	1.015 [0.998, 1.032]	0.999 [0.909, 1.098]	0.034 [0.034, 0.034]†	0.035 [0.035, 0.035]†
16	1	0	missing	missing	missing	missing

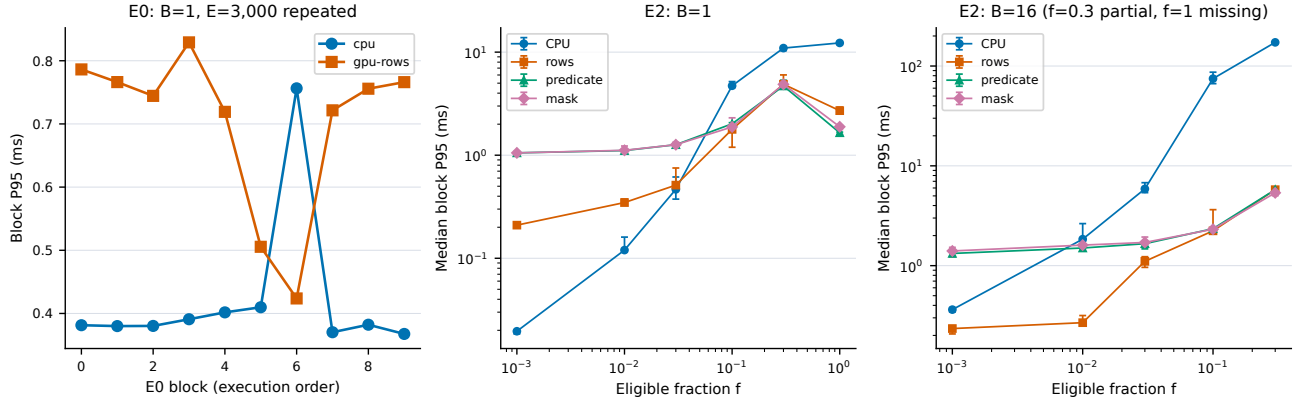


Figure 7: E0/E2 partial archive e4ca1336, RTX 4090/Vulkan, 100K×384 AG News, $k = 10$. Left: E0 block P95 in execution order; no block removed. Middle and right: E2 median block P95 with 95% block-bootstrap intervals, batch-call latency. Missing cells are absent, not interpolated; $B = 16$, $f = 0.3$ uses one CPU and two GPU blocks. Logarithmic axes.

In sparse cells, the eligibility representation is the largest lever measured. At $f \leq 0.01$, compact rows are 3.2–6.1× faster than shader predicates and dense masks at both batch sizes, for P50 and P95, and every such interval clears the conservative E0 screen. The advantage narrows through $f = 0.03$ and 0.1 and is unresolved at 0.3. At $B = 1$ and $f = 1$, the predicate is faster (rows/predicate P95 1.673 [1.563, 1.796]); that effect does not clear the E0 screen. Predicate and mask are close to each other throughout. The transition between rows and predicate for $B = 1$ is therefore bracketed only by (0.1, 1), with no support for interpolating through 0.3; for $B = 16$ no transition estimate exists, because 0.3 has two blocks and 1.0 is missing.

Batch moves the CPU/GPU winner by more than an order of magnitude in E on one host. At $B = 1$, CPU beats compact rows by 10.9× at $f = 0.001$ and 2.6× at 0.01 (P50), the 0.03 cell is unresolved at P95, and rows

win from 0.1. At $B = 16$, rows already win at $E = 100$ (P50 0.709 [0.705, 0.725]) and are about $29\times$ faster at $f = 0.1$; batch-16 CPU latency grows nearly linearly with batch, whereas the GPU amortizes. The $B = 1$, $E = 3,000$ interval (rows/CPU P95 1.211 [0.834, 2.004]) is too wide to locate a scalar threshold.

9.4 A same-host ordering witness

Table 8: CPU versus compact-row WGPU at two cells from one host and binary. Ratio is rows/CPU (above one favors CPU) with a 95% paired block bootstrap. E0 and E2 blocks at $B = 1$, $E = 3,000$ are separate phases of the same campaign. Post-hoc use of preregistered cells.

Phase	B	E	EDB	Metric	Blocks	CPU faster	Rows/CPU [95% CI]
E0	1	3,000	1,152,000	P50	10	9	1.921 [1.439, 1.963]
E0	1	3,000	1,152,000	P95	10	9	1.968 [1.550, 2.062]
E2	1	3,000	1,152,000	P50	5	5	1.209 [1.020, 2.004]
E2	1	3,000	1,152,000	P95	5	3	1.211 [0.834, 2.004]
E2	16	100	614,400	P50	5	0	0.709 [0.705, 0.725]
E2	16	100	614,400	P95	5	0	0.661 [0.554, 0.729]

Table 8 replicates R1’s ordering contradiction without relying on a near-tie. With $D = 384$ and $k = 10$ fixed, WGPU wins at $EDB = 614,400$ in 5 of 5 blocks, while CPU wins at the larger $EDB = 1,152,000$ in 9 of 10 E0 blocks at both percentiles and in all five E2 blocks at P50. No increasing EDB threshold on backend choice classifies both cells. Two qualifications remain: the witness was selected after the fact from cells designed for other questions, and the E0 ratio of about $1.9\times$ does not clear the same-engine A/A band, which is itself about $1.9\text{--}2\times$. The E2 blocks of the same cell give a smaller ratio of 1.21, consistent with the GPU-state drift above.

9.5 Mechanism diagnostics

Table 9: E2 $B = 1$ per-call medians across blocks: completed call, host row materialization, device-timestamp selection and scoring (ms), and bytes uploaded per call. Device timestamps overlap host phases and are not additive. Upload values at $B = 1$ are unaffected by the harness defect described below.

E	Representation	Call	Materialize	Select	Score	Upload B
100	rows	0.185	0.005	0.018	0.006	2,000
100	predicate	0.996	0.007	0.704	0.023	401,600
100	mask	1.025	0.007	0.704	0.021	401,600
3,000	rows	0.445	0.141	0.133	0.012	13,600
3,000	predicate	1.160	0.175	0.711	0.026	401,600
3,000	mask	1.211	0.180	0.711	0.025	401,600
30,000	rows	4.333	2.206	1.183	0.106	121,600
30,000	predicate	4.347	2.240	1.376	0.109	401,600
30,000	mask	3.341	1.742	1.162	0.081	401,600
100,000	rows	2.549	0.312	1.044	0.167	401,600
100,000	predicate	1.537	0.311	0.696	0.168	401,600
100,000	mask	1.691	0.323	0.696	0.167	401,600

Per-call diagnostics explain part of the representation gap and expose avoidable work. Predicate and mask selection runs over all 100K rows and takes about 0.70 ms at every measured E except 30K (1.16–1.38 ms), whereas compact-row selection scales with E (0.018 ms at $E = 100$). Both predicate and mask upload 400,000 bytes per call even when 100 rows are eligible: the mask as a 32-bit word per row, the predicate as the chunk liveness vector. In the measured binary, all three GPU representations also pay host row materialization because the eligibility plan retains rows for routing and fallback; at $E = 30K$ it is 2.206 ms of a 4.333 ms compact-row call (51%), compared with 0.312 ms at $E = 100K$. Source inspection shows the reason: an all-rows predicate is a sequential slot scan, whereas a timestamp range walks an ordered index of per-timestamp sets and then sorts the collected slots. This is why the $E = 30K$ GPU cells are slower than $E = 100K$. The audited source now removes

that unused list from required shader-predicate batches and reports that no host rows were materialized, but no comparable latency rerun exists. At full eligibility, compact-row selection is slower than predicate selection over the same 100K rows (1.044 versus 0.696 ms), which is consistent with the predicate’s dense advantage. These attributions are hypotheses supported by overlapping counters and code, not a causal decomposition.

A harness defect in batch counters. Each response of one native GPU batch carries batch-total transfer counters, as the library documents, but the benchmark summed them over responses. Batch- B upload, readback, and lock-wait fields were therefore overstated B -fold. The E2 batch-16 upload of 6,794,240 bytes is exactly $16 \times (24,576 + 400,000 + 64)$; the true per-call value is 424,640. The same defect affected the S2 batch-eight figures in Section 12. Latency, recall, allocation, and phase timings are unaffected. The harness now records each batch total once and fails if responses disagree; retained raw fields are not rewritten.

9.6 Impact of missing cells

The missing $B = 16$ cells remove the high-fraction endpoint for batch 16 and leave its $f = 0.3$ comparisons with $n = 2$ (one CPU block). Consequently the batch-16 representation transition, the batch-16 dense predicate-versus-rows ordering, and any batch-16 CPU/GPU statement above $f = 0.1$ remain unmeasured. The $B = 1$ conclusions, the sparse representation effect at both batches, and the ordering witness use only complete cells. No missing value is imputed, and no cell is described as complete.

10 Predicate representation and batching: historical context

H0 dense GPU endpoints share a recorded GPU revision: shader predicates take 3.2404 ms P95, compact rows 4.2428, and masks 4.4591. At 1K eligible rows, compact rows take 0.6766 ms and shader predicates 2.8832 ms, but the sparse records differ in revision. That difference is context, not a causal representation ablation. The CPU and dense-GPU endpoints also differ. Figure 8 preserves these limitations rather than merging H0 with H1.

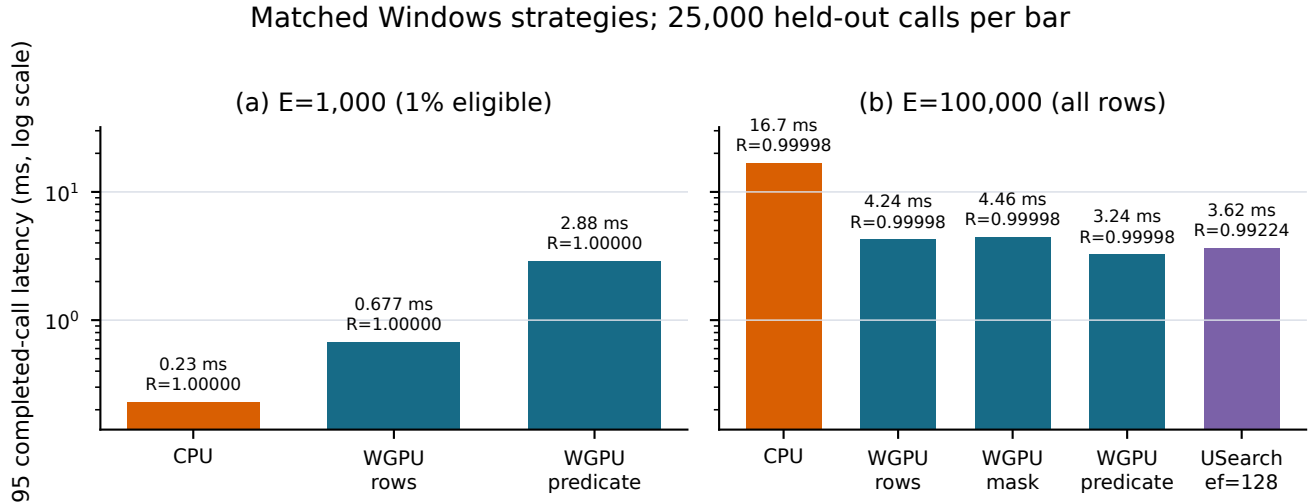


Figure 8: H0 RTX 4050/DX12, 100K×384 AG News, $B = 1$, $k = 10$, five runs/25,000 calls per row, completed-call P95. Dense WGPU forms share `f1ccc4d`; CPU/sparse records include other revisions. Exhaustive sparse/dense FP64 recall 1.0/0.99998; USearch ANN 0.99224. Descriptive context; no causal cross-revision speedup.

Batching affects dispatch amortization and selection; representation affects host preparation and device work. C1–C6 supports both as routing features but changes them together with dimension and selectivity. Device-lab batches use another reporting schema. E2 (Section 9) is the first same-host, same-binary representation-by-batch-by-eligibility ablation; its sparse compact-row advantage agrees in direction with H0’s cross-revision 1K endpoint, and its dense predicate advantage with H0’s dense endpoint. E2 still fixes N , D , k , the corpus, and the predicate shape, and no fixed- E , varying- N study establishes eligibility as a statistically superior predictor to size.

The reference C1 and C5 cells sharpen this boundary without isolating it. Both have $E = 1K$, $D = 128$, $B = 1$, and $k = 10$, hence equal EDB . C1 is a 1K dense collection; C5 obtains the same eligible count through a one-percent predicate over 100K rows. Current CPU P95 rises from 0.0143 to 0.0341 ms, while candidate WGPU remains near 0.094–0.095 ms. Total N and predicate preparation change together, so no causal coefficient follows, but equal eligible scoring work plainly does not imply equal completed-call cost.

11 ANN and external-engine comparisons

11.1 Chroma qualification and historical competitors

Seven qualified S1 pairs favor candidate WGPU over Chroma HNSW, with P95 ratios from 3.700 to 3233.136. All seven paired Chroma rows have recall 1.0. The largest ratio is the real AG News workload: Chroma takes 555.964 ms versus WGPU’s 0.171958 ms. Three attempts fail the gate before held-out timing: C3 and C6 at expansion 128, and deep D2 at 512. Completed USearch D2 has recall 0.3453125 at 83.3133 ms and is unqualified. These rows remain visible.

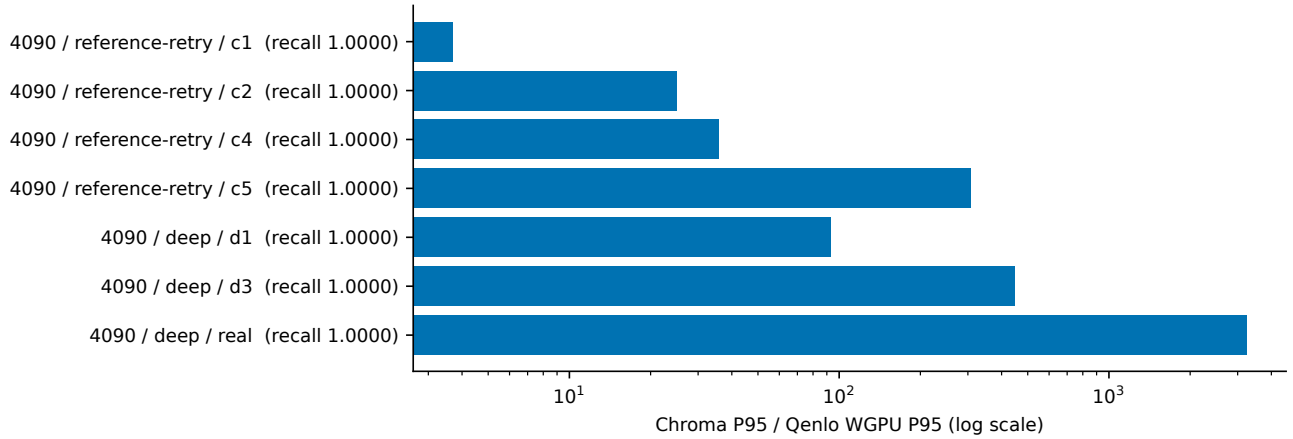


Figure 9: S1 730c0500, seven qualified configuration/workload pairs on A40/RTX 4090, $k = 10$; IDs map to matrix/appendix. Logarithmic axis: Chroma/native candidate-WGPU completed-call P95 ratio. Chroma recall printed. Python API, ANN, persistence/filter boundaries differ. Three gate failures excluded from ratios, retained in ledger; no error bars.

H0 Chroma dense reaches 1.1303 ms P95 at recall 0.99414, compared with 3.2404 ms for WGPU predicate evaluation at recall 0.99998. Tuned USearch with expansion 128 takes 3.6245 ms at recall 0.99224. These are operating points with revision and API qualifications, not ANN frontiers. In H3, Linux FAISS HNSW takes 1.4939 ms at recall 0.9994, compared with 12.9014 ms and recall 1.0 for Qenlo CPU. Chroma, USearch, FAISS Flat, Milvus Lite, and LanceDB remain in the appendix. These differences preclude universal rankings.

11.2 A6000 real and synthetic evidence

The H4 strict TopK predicate yields $E = 10,026$ from 1M 768-dimensional rows. FAISS GPU Flat leads the qualified methods at 0.132222 ms P95, followed by the research CUDA prototype at 0.170722 ms and NumPy at 0.197311 ms. Each has 5,000 calls and FP64 recall 1.0. cuVS’s 0.384636 ms row is disqualified because recall is 0.9. The prototype also misses the retained gate requiring a twofold CPU advantage. Eligibility is pre-materialized, and the prototype is Python/PyTorch CUDA rather than shipped Rust/WGPU.

The fresh-seed H5 synthetic cohort differs. FAISS leads at $E = 1K$ and 10K; the buffered prototype leads at 100K (0.5673 versus 0.7887 ms) and 1M (4.6683 versus 6.3475 ms). The reversal lies between the measured 10K and 100K points. Correctness is unordered top-10 agreement rather than an FP64 oracle. This result is specific to the formulation. Native WGPU has no usable A6000 Vulkan ICD and therefore no native timing in this cohort.

12 Mutation, reopen, and resource accounting

The S1 lifecycle uses a durable $10K \times 384$ collection on an RTX 4090 with $B = 1$, $k = 10$, and no filter. First-search P95 after add-one, delete-one, add-batch, and delete-batch is 0.598, 0.631, 0.740, and 0.819 ms, respectively. Mutation is excluded and has a separate P95 near 22 ms. Three observations of each mutation type report zero rebuilds, and immediate-deletion checks pass. Reopen takes 52.169 ms to open and, separately, 101.296 ms for the first search with one rebuild. Neither value is a combined timer. One observation cannot characterize reopen tail latency.

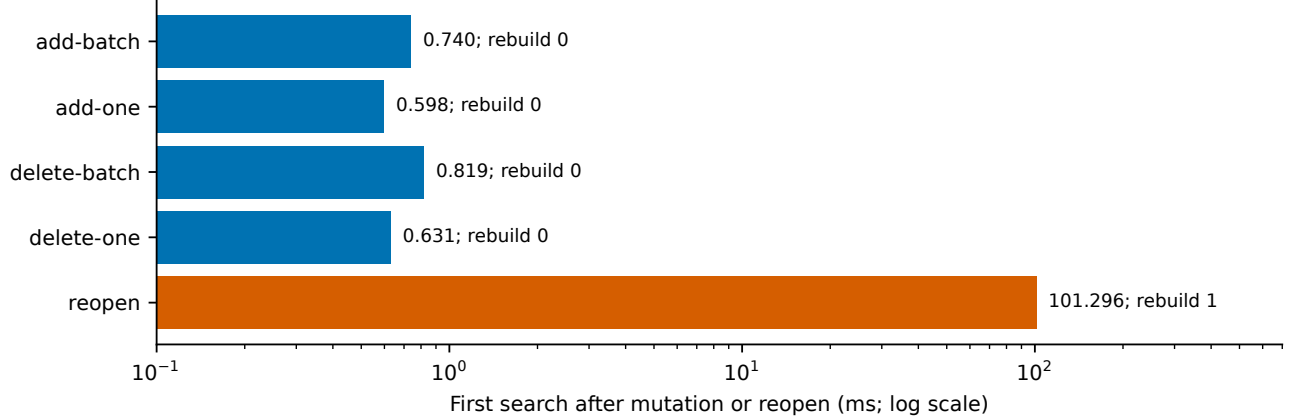


Figure 10: S1 reference RTX 4090, durable $10K \times 384$, unfiltered $B = 1$, $k = 10$. Bars are first-search latency after mutation/reopen, excluding mutation/opening. Three observations per mutation type give P95; reopen is one observation. Rebuild annotations are fractions, not probabilities. Deletion checks pass. Logarithmic horizontal axis, no error bars.

S2 unfiltered WGPU reports 311.181 MB of owned accelerator allocation and a 1.203675 GB process RSS high-water mark. Batch eight reports 314.909 MB and 1.174577 GB, respectively. Units are decimal. RSS includes process and runtime peaks; owned buffers omit driver and runtime allocations. Neither is physical VRAM telemetry. Unfiltered batch-one upload and readback are 403,136 and 48 bytes per call. For batch eight the retained fields read 517,120 and 65,792 bytes, exactly eight times 64,640 and 8,224; the harness summed a batch total repeated on each response (Section 9). The per-call transfer implied by the code, 24,576 query bytes, 40,000 row-ID bytes, and 64 parameter bytes up and 8×64 16-byte candidates plus 32 timestamp bytes back, is 64,640 and 8,224 bytes. Residency and per-call transfer are separate scopes.

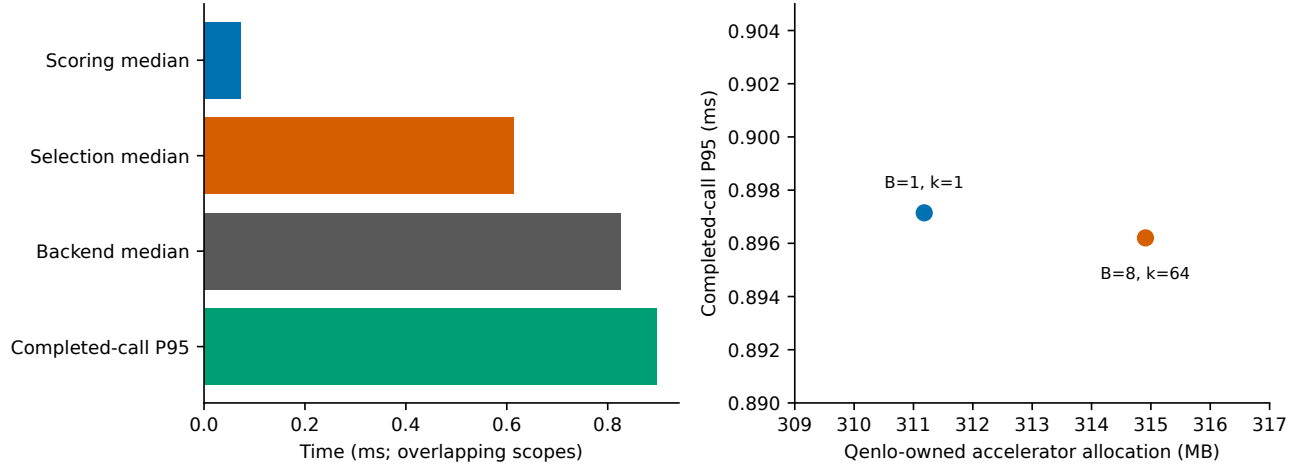


Figure 11: S2 `bb195fed`, RTX 4090/Vulkan, generated $100K \times 768$, FP64 recall 1.0. Left: $B = 8, k = 64, E/N = 10\%$, raw phase medians versus call P95. Nested scopes are not additive. Right: two corrected WGPU workloads, zoomed linear latency axis, decimal owned-allocation MB. Different workloads do not establish a memory/latency law or efficiency score. No error bars.

Batch-eight median scoring is 0.072 ms, selection 0.613, backend execution 0.826; call P95 is 0.896. Selection exceeds scoring in these diagnostics. That motivates investigation but does not prove another selector improves latency. Capacity, initialization, and warm-state validity remain deployment costs even when arithmetic is inexpensive.

13 Mobile and heterogeneous devices

H6 records physical Android: Android 16 (API 36), MediaTek MT6897, Mali-G615 MC6 with Vulkan, app 0.1.0. All 21 aggregate cells report $\text{recall@10}=1.0$ without fallback or failure. CPU wins the 10K quick batch-one cell (5.767 versus WGPU's 12.487 ms P95); WGPU wins the 100K full cell (16.975 versus 29.436) and soak cell (17.678 versus 27.019). Size and suite change together, so this is a qualitative reversal rather than a mobile crossover bracket.

Android 16 / Mali-G615 device-lab evidence; observed $\text{recall@10} = 1.0$

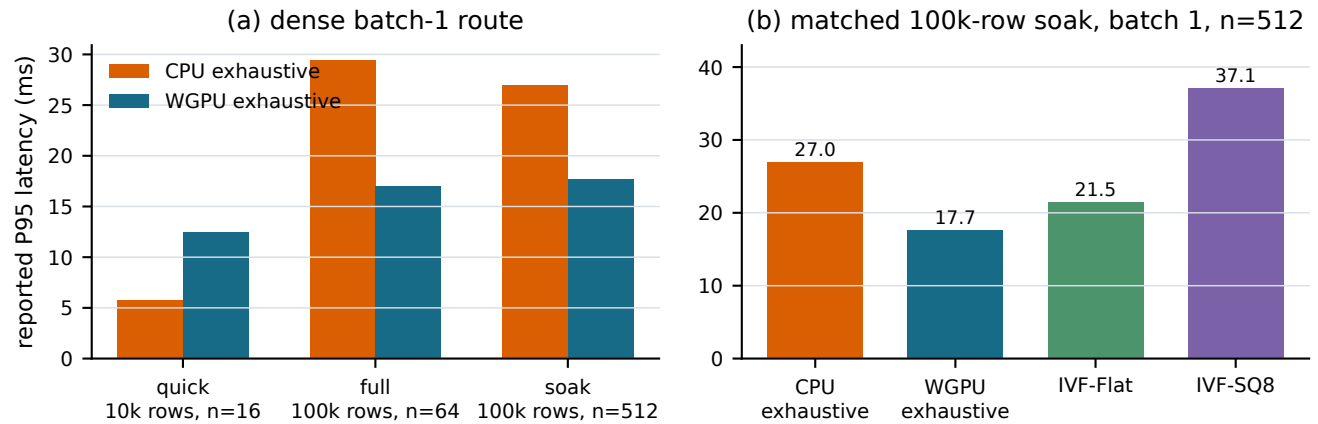


Figure 12: H6 physical Mali-G615/Vulkan, generated 384D, $k = 10$, app 0.1.0; commit unrecorded. Supplied aggregate percentiles: quick/full/soak use 16/64/512 samples/cell. All recalls 1.0; IVF remains approximate. Raw calls and controlled thermal/power records absent. No error bars or current-revision mobile claim.

At the 100K Android soak point, IVF-Flat takes 21.453 ms P95 and IVF-SQ8 takes 37.090 ms, both slower than exhaustive WGPU at those settings. Dense WGPU batch eight reports 11.131 ms; this batch statistic is not normalized per query. It suggests amortization, not an eightfold per-query gain. The power source is absent, and the thermal state is the uninterpreted value “0”.

The H7 Intel Arc soak reports CPU at 16.486 ms P95, exhaustive WGPU at 4.444 ms, IVF-Flat at 2.704 ms, and IVF-SQ8 at 22.797 ms for 100K×384. All recalls are 1.0 over 512 samples per cell. Arc adds non-NVIDIA execution evidence and another negative result for universal exhaustive dominance: IVF-Flat wins this cell. One externally labeled “full” report contains `suite=quick`; the JSON is authoritative. Historical Android runs and Kotlin/JVM bindings do not establish current Android packaging. Simulator and macOS builds do not establish physical iOS performance, and no iOS-device or MPS timing is retained.

14 Negative results and rejected optimization

Lane-minimum selection retains each lane’s minimum and rescans only the winning lane. It reduces repeated scans but changes local state and synchronization. Twelve qualified S0/S1 pairs yield five wins and seven losses, with frozen-baseline to candidate P95 ratios from 0.617 to 1.295. The retained implementation uses the simpler full-rescan selector. Less nominal work did not consistently reduce completed latency. Archives contain other changes too: paired revisions do not prove every difference is solely caused by selection.

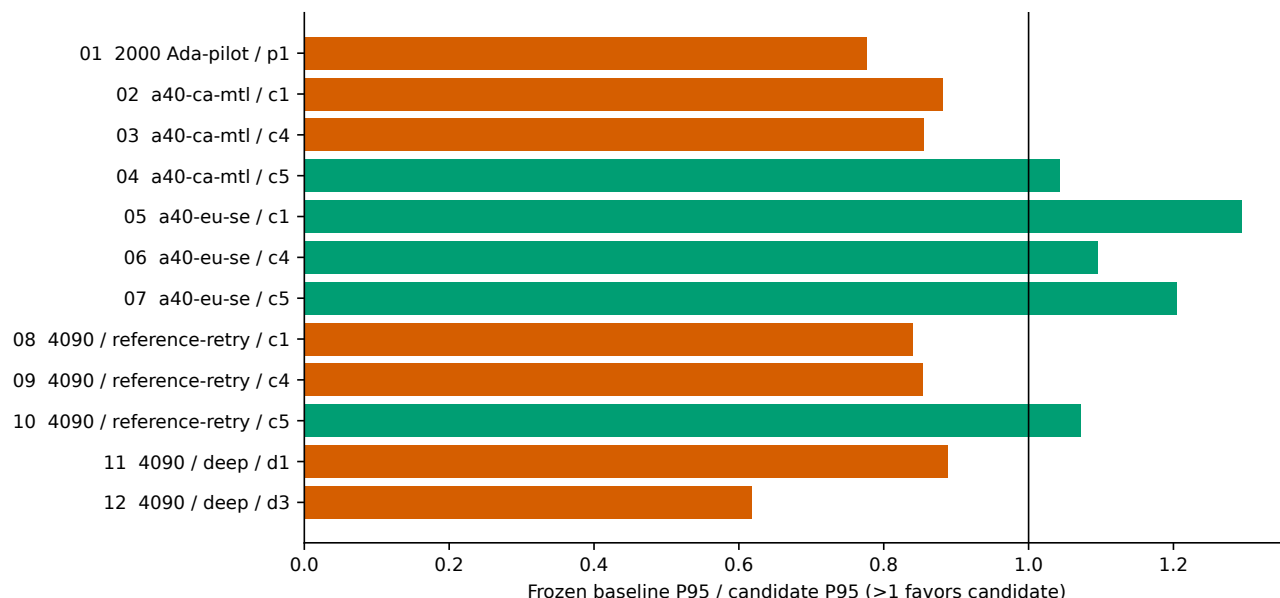


Figure 13: S0 f37e744c versus S1 730c0500, twelve qualified WGPU configuration/workload pairs, FP64-oracle-qualified completed-call P95. Exact workloads/ratios in appendix. Above one favors candidate: five wins, seven losses. RTX 2000 Ada, A40, RTX 4090; no significance test, error bars, or pooled device effect.

Four pre-fix 100K×768 failures remain because a 2 GiB benchmark budget did not override the 512 MiB default in-memory admission limit. The corrected source and result archives have distinct hashes; correction does not retroactively qualify the failed rows. Two malformed workload-label invocations are invalid-harness rows, not slow engine results. Chroma gate failures, unqualified USearch, cuVS recall or runtime failures, and unavailable Vulkan paths remain part of the evidence. A controller reaching completion is insufficient for qualification.

R1 is the negative routing result the earlier archive lacked. The frozen *EDB* candidate passed development and failed held out; it was reverted before release. P2 similarly rejects unconditional use of a sparse CPU optimization. Together with the five-win/seven-loss WGPU selector, these results show three distinct ways that plausible work reduction can fail at the completed-call boundary. The E0/E2 audit adds a measurement-integrity negative: batch transfer counters in the harness were overstated by the batch size, and an exit-code failure entered the data through a resume rule. Both are disclosed rather than silently repaired; the runner now requires a zero exit record as well as a passing summary before it resumes past a destination.

Final known daily Runpod spend is \$0.9400778694252952. Zero campaign pods remain, with ten creations and ten verified deletions. No campaign volume was retained. This is account and daily billing, not per-query cost or a clean marginal campaign cost; billing lag is possible. No cloud work was rerun for authoring.

15 Deployment implications

CPU fits tiny eligible workloads when accelerator fixed costs dominate. WGPU can help dense, batched workloads when residency is valid. Tensor execution may fit applications that already own its runtime, but package footprint and persistence-integration costs were not measured. Before preparation is amortized, callers must account for snapshot generation and mutation state. An explanatory decomposition is

$$T_{\text{call}} = T_{\text{prepare}} + T_{\text{route}} + T_{\text{execute}} + T_{\text{return}}. \quad (4)$$

These conceptual, nonoverlapping boundaries are not sums of overlapping counters. Candidate explanatory variables include E , D , B , k , representation, and device; R1 rejects one policy based on their product, but does not isolate each factor’s causal or predictive contribution. Preparation also depends on total N , predicate shape, reuse, generation, and residency. No coefficients are fitted. The decomposition organizes the observations without asserting a transferable latency law.

The reversals and preparation ablation support the central engineering inference that collection size or GPU availability alone is insufficient for routing. R1 goes further: one specific multiplicative predictor is rejected on held-out data. E0/E2 adds what a router must actually decide and measure.

- *Representation is part of the route.* At sparse eligibility on the RTX 4090, choosing shader predicates instead of compact rows costs $3.2\text{--}6.1\times$, more than most CPU/GPU gaps near crossover. The benchmark’s automatic arm, like R1’s, uses shader predicates; at $B = 16$, $E = 100$ its forced predicate latency is 1.209 ms versus 0.198 ms for rows and 0.278 ms for CPU (P50). The automatic path itself was not measured in E2, so this is an expected-regret hypothesis, not an observed automatic latency.
- *Preparation belongs in the cost model.* Host row materialization can exceed half of a GPU call at mid selectivity, and its cost depends on predicate shape, not only on E .
- *Certification must be noise-calibrated.* Block-replicated, interleaved forced and automatic arms with clock telemetry are required, and regret limits must exceed the measured A/A band or be tested with replicated paired blocks.

A candidate policy should preserve these variables rather than collapse them, bind calibration to source, device, and API identity, include a confidence-aware fallback, and evaluate realized automatic latency after mutations and cold preparation. These requirements define the next experiment; they are not evidence that a learned or calibrated router will succeed. No threshold from this partial, single-host campaign is deployed, and cloud results cannot select a mobile default.

16 Threats to validity

The archive grew sequentially rather than as a randomized factorial study. Dirty trees limit attribution even when hashes and configurations are retained. H0 is cross-revision context. H1 is a post-hoc study on one laptop, one dimension, one batch size, and one corpus, with independently generated metadata. P0 differs from H1 in several environment and source variables; opposite winners falsify universality but do not identify which variable moved the boundary. H2 is sequential, lacks CPU cells, and cannot recalibrate H1. C1–C6 confound collection size, dimension, batching, and eligibility. S2 uses a rejected selector whose final replacement was not rerun in the corrected cells.

E0/E2 is partial (227 of 260 summaries), ran on one host, and measured noise at one cell; its five-block intervals are coarse, two-block intervals are weak, and bootstrap endpoints depend on the resampling seed in the third decimal. One included block exited with code 101, and GPU clocks were sampled only before and after each process. The ordering witness is post hoc. Its benchmark binary and dataset hashes are attested by the manifest rather than recomputed from the archive.

The three-run small campaign, five-run historical measurements, and single-reopen observation do not support narrow intervals or broad significance. R1 is held out relative to the selected rule, but its sixteen designed cells

are not a random deployment distribution, its per-cell labels are sensitive to run variation, and its post-hoc alternative threshold must not be promoted. A harness defect overstated batch transfer counters; other retained batch-greater-than-one transfer fields produced by the same harness should be read as B times the per-call value. Repeated queries are not independent deployment workloads. Aggregate device data cannot yield raw distributions, and thermal and power controls are incomplete. Prototype agreement is weaker than an FP64 oracle. Reference checks do not prove that all ties or workloads agree.

Adapters differ in filtering, selection, normalization, threading, persistence, and API. NumPy uses full lexicographic sorting rather than an optimal top- k baseline. Corrected FAISS is CPU Flat, whereas the historical A6000 result is GPU Flat. Memory scopes differ and omit peak total accelerator use. Concurrency, energy, device loss, cache churn, adversarial predicates, and crash schedules were not benchmarked. Native sweeps lack ANN frontiers and filter-aware baselines. No fastest-database claim follows.

17 Related work and novelty boundary

FAISS describes optimized exhaustive and approximate CPU and GPU search and selection [3, 6]. HNSW provides graph-based ANN search [8]. DiskANN and SPANN address storage-aware regimes [2, 12]; Filtered-DiskANN and ACORN modify graph access under predicates [4, 10].

Filtered-ANN studies establish that attributes, selectivity, query and data conditions, tuning, and implementation affect method choice [5, 7, 11]. System analyses examine filtering plans and approximate and exhaustive alternatives [1]; the latter two studies are preprints. The observation that selective scans sometimes win is therefore not novel. Qenlo’s narrower contribution is the completed-call CPU and portable-GPU boundary, eligibility compilation with generation-bound state, a same-host measurement of how eligibility representation and repeat-block noise interact with that boundary, and resource and lifecycle evidence for small collections. The archive lacks broad FANNS coverage and claims no new ANN algorithm.

18 Conclusions

The principal confirmatory result is the rejection of the frozen *EDB* rule: zero errors on 31 development pairs became 8 errors in 16 held-out workloads, with 235.7% maximum backend-choice regret. Separately, an *E*-only threshold cannot classify opposite observed winners at the same 3K and 4K eligible counts across two environments. R1’s own ordering contradiction depends on a 3.16%-margin cell, but a block-replicated same-host witness now shows the same contradiction with 9 of 10 noise-control blocks on the CPU side. A post-hoc scalar threshold still stays below the original regret limits on R1, and R1’s count of wrong choices is sensitive to run variation. These observations do not prove that every scalar policy is practically inadequate or that query-shape factorization caused the differences.

The positive conclusion is a routing architecture, not a fitted policy. Execution decisions must preserve E, D, B, k , total N , predicate shape and representation, source/device/runtime identity, and warm or mutation state. On one RTX 4090 host, representation changes sparse completed-call latency by $3.2\text{--}6.1\times$, batch moves the CPU/GPU winner by more than an order of magnitude in E , host materialization can dominate a GPU call, and same-cell repeat blocks differ by up to about $2\times$. Eligibility plans reduce avoidable preparation; the audited implementation now removes unused host-row materialization from required predicate batches, while sparse-only CPU optimization and GPU selection must themselves be routed. Automatic-path latency must be measured directly because it is not interchangeable with forced-route P95. The archive also establishes scoped engineering opportunities: the dense Qenlo CPU path trails single-thread FAISS Flat in P2, WGPU selection can exceed scoring in the corrected 768D diagnostics, and predicate and mask modes still select over and upload per-row state for every call.

No retained result demonstrates that a multidimensional production router generalizes. The next valid claim requires completing the 33 missing E2 slots, a frozen policy that includes representation, interleaved block-replicated forced-route and automatic measurements with clock telemetry, disjoint held-out devices and query shapes, cold/warm and post-mutation strata, and regret limits calibrated to the measured noise. Until that experiment passes, Qenlo has evidence for conditional execution and calibrated fallbacks, not for an adaptive-router advantage or a portable performance promise.

Reproducibility statement

Every measurement in this paper comes from retained evidence; no cloud experiment was rerun for authoring. All E0/E2 measurements predate four source changes made during this audit: the benchmark records batch-total counters once per batch (run format v4); timestamp-range row materialization switches from index traversal to a sequential scan after a bounded probe; required shader-predicate batches count eligibility without building an unused host row list; and the campaign runner refuses to resume past a summary whose run record is absent or nonzero. None is reflected in reported campaign latency, and no production routing rule changed. The E0/E2 reductions are reproduced from the retained tarball by the scripts named in Appendix B. The archive-wide script verifies four source archives, regenerates the new reductions, and inventories every discovered tracked or selected archive-internal sample series. Ledgers preserve claims, revisions and hashes, environments, provenance, timing, counts, correctness, memory, qualification, and exclusions. Paper-only scripts regenerate reductions, tables, and PDF/PNG figures without replacing raw evidence or historical figures. Appendix B and `paper/REPRODUCE.md` describe the audit and clean build.

References

- [1] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis. *arXiv preprint arXiv:2602.11443*, 2026. URL <https://arxiv.org/abs/2602.11443>.
- [2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Advances in Neural Information Processing Systems*, volume 34, pages 5199–5212, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html.
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL <https://arxiv.org/abs/2401.08281>.
- [4] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416. Association for Computing Machinery, 2023. doi: 10.1145/3543507.3583552. URL <https://doi.org/10.1145/3543507.3583552>.
- [5] Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoefer. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 610–622, 2026. doi: 10.1145/3805712.3809731. URL <https://doi.org/10.1145/3805712.3809731>.
- [6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572. URL <https://doi.org/10.1109/TBDATA.2019.2921572>.
- [7] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proceedings of the ACM on Management of Data*, 3(6):298:1–298:26, 2025. doi: 10.1145/3769763. URL <https://doi.org/10.1145/3769763>.
- [8] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [9] Beatrix Miranda Ginn Nielsen. DTU pretrained sentence BERT AG News embeddings, 2023. URL <https://doi.org/10.11583/DTU.21286923.v1>.

- [10] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3): 120:1–120:27, 2024. doi: 10.1145/3654923. URL <https://doi.org/10.1145/3654923>.
- [11] Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study. *arXiv preprint arXiv:2509.07789*, 2025. URL <https://arxiv.org/abs/2509.07789>.
- [12] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>.
- [13] TopK. TopK Bench: Vector database benchmarks. <https://github.com/topk-io/bench>, 2026. Commit 398ce7d72dea7ad765ada54c5daa014c498efb52; accessed 2026-09-05.
- [14] W3C GPU for the Web Working Group. WebGPU. <https://www.w3.org/TR/webgpu/>, 2026. W3C Candidate Recommendation Draft, 20 August 2026; accessed 2026-09-05.
- [15] W3C GPU for the Web Working Group. WebGPU Shading Language. <https://www.w3.org/TR/WGSL/>, 2026. W3C Candidate Recommendation Draft, 17 August 2026; accessed 2026-09-05.

A Detailed retained tables

These tables are generated from retained machine-readable evidence. They preserve aggregation within each cohort, and an empty field is not a zero. The full 182-row campaign CSV and JSON include every failed, unavailable, invalid, and unqualified row. The main table explains the difference between native-call and replay sample units. All latency units below are milliseconds.

A.1 Common small-collection native rows

Table 10: S1 common cells from the matrix. CPU values on A40 are absent for this source role; no substitute baseline-CPU row is silently inserted. Workload definitions appear in the main text. All displayed rows qualify at FP64 recall 1.0.

Configuration	Cell	CPU P95	WGPU P95
a40-ca-mtl	C1	–	0.1595
a40-ca-mtl	C2	–	0.2398
a40-ca-mtl	C3	–	0.4277
a40-ca-mtl	C4	–	0.1514
a40-ca-mtl	C5	–	0.1590
a40-ca-mtl	C6	–	13.7685
a40-eu-se	C1	–	0.1779
a40-eu-se	C2	–	0.3574
a40-eu-se	C3	–	0.4330
a40-eu-se	C4	–	0.1582
a40-eu-se	C5	–	0.1666
a40-eu-se	C6	–	13.7465
4090 EU-RO	C1	0.0143	0.0943
4090 EU-RO	C2	0.0484	0.1347
4090 EU-RO	C3	1.9988	0.2695
4090 EU-RO	C4	0.0082	0.0911
4090 EU-RO	C5	0.0341	0.0953
4090 EU-RO	C6	68.0575	4.0479

A.2 Frozen-baseline versus candidate selection

Table 11: All twelve qualified S0/S1 pairs. The ratio is frozen-baseline P95 divided by candidate P95. IDs encode row count (r), dimension (d), batch size (b), k when nondefault, and eligible fraction (f). The compound cell combines metadata clauses.

Configuration	Workload	Ratio
rtx2000-eu-ro-pilot	p1-r1000-d128-b1-f1	0.777
a40-ca-mtl	c1-r1000-d128-b1-f1	0.881
a40-ca-mtl	c4-r10000-d384-b1-f001	0.855
a40-ca-mtl	c5-r100000-d128-b1-f001	1.043
a40-eu-se	c1-r1000-d128-b1-f1	1.295
a40-eu-se	c4-r10000-d384-b1-f001	1.096
a40-eu-se	c5-r100000-d128-b1-f001	1.205
4090 / reference-retry	c1-r1000-d128-b1-f1	0.840
4090 / reference-retry	c4-r10000-d384-b1-f001	0.853
4090 / reference-retry	c5-r100000-d128-b1-f001	1.073
4090 / deep	d1-r5000-d384-b8-k10-f01	0.889
4090 / deep	d3-r50000-d384-b1-k10-compound	0.617

A.3 Small real-embedding replay

Table 12: S1 AG News on the reference RTX 4090, $100K \times 384$, $B = 16, k = 10, E/N = 1\%$. There are three runs and 939 calls per native or replayed engine. Chroma and USearch are ANN; all other engines are exhaustive. External flat and tensor matrices are prefiltered.

Engine	Completed-call P95	FP64 recall
chroma-ef512	555.9637	1.00000
current-cpu	0.9416	1.00000
current-gpu	0.1720	1.00000
faiss-flat	0.2427	1.00000
numpy	0.7492	1.00000
torch-cpu	0.2683	1.00000
torch-cuda	0.3514	1.00000
usearch	294.6324	0.99994

A.4 Historical Windows endpoints and matched localization

Table 13: H0: $100K \times 384$, $B = 1, k = 10$, five runs and 25,000 calls per row. Cross-revision endpoint context: dense CPU **4ae9f1d**, dense WGPU **f1ccc4d**; sparse CPU and rows **78e3947**, sparse predicate **f1ccc4d**. ANN rows retain their own revisions and APIs.

Eligible	System	P95	FP64 recall
1000	CPU exhaustive	0.2304	1.00000
1000	WGPU compact rows	0.6766	1.00000
1000	WGPU predicate	2.8832	1.00000
100000	CPU exhaustive	16.6567	0.99998
100000	WGPU compact rows	4.2428	0.99998
100000	WGPU mask	4.4591	0.99998
100000	WGPU predicate	3.2404	0.99998
100000	USearch HNSW ef=128	3.6245	0.99224
100000	Chroma HNSW ef=128	1.1303	0.99414

Table 14: H1: matched recorded revision **3e2a4a9**, RTX 4050/DX12, AG News, $100K \times 384$, $B = 1, k = 10$, five runs and 25,000 calls per backend and cell. Post-hoc localization; median run P95.

Eligible	CPU P95	Compact WGPU P95	FP64 recall
2000	0.5198	0.6340	1.00000
3000	0.9563	0.8144	1.00000
4000	1.3305	1.0737	1.00000
6000	1.9735	1.6033	1.00000
8000	2.5003	2.1508	1.00000
10000	2.8104	2.6041	1.00000

A.5 Historical eligibility ablation

Table 15: H2, RTX 4090/Vulkan, 100K×384, $B = 1, k = 10$. Five runs and 25,000 calls per mode and cell, with modes executed sequentially. Percentiles are medians of run percentiles.

Eligible	Preparation	P50	P95	P99	FP64 recall
1000	legacy-two-pass	0.1204	0.1317	0.1584	1.00000
1000	one-pass	0.1097	0.1179	0.1453	1.00000
1000	cached	0.0908	0.0997	0.1132	1.00000
4000	legacy-two-pass	0.2910	0.3053	0.3251	1.00000
4000	one-pass	0.1982	0.2080	0.2250	1.00000
4000	cached	0.1348	0.1438	0.1582	1.00000
6000	legacy-two-pass	0.3851	0.4011	0.4400	1.00000
6000	one-pass	0.2561	0.2716	0.2903	1.00000
6000	cached	0.1624	0.1713	0.1941	1.00000
100000	legacy-two-pass	1.6633	1.7862	1.9031	0.99998
100000	one-pass	1.6074	1.7205	1.8165	0.99998
100000	cached	1.5379	1.5960	1.6907	0.99998

A.6 Archive-wide routing and CPU reductions

Table 16: R1 all sixteen held-out workloads for the frozen $EDB = 10^6$ rule. Work is in millions; regret compares the selected forced route with the faster forced route. Auto/best compares the separately measured automatic P95 with the faster forced P95 and is not interpreted as overhead.

Workload	$EDB/10^6$	k	Winner	Choice	Regret %	Auto/best
d128-b8-e488-k1	0.500	1	CPU	CPU	0.0	0.727
d128-b8-e733-k10	0.751	10	CPU	CPU	0.0	0.821
d128-b8-e977-k64	1.000	64	CPU	GPU	41.6	3.289
d128-b8-e1221-k10	1.250	10	CPU	GPU	235.7	4.021
d128-b8-e1465-k1	1.500	1	GPU	GPU	0.0	2.394
d384-b1-e1302-k1	0.500	1	CPU	CPU	0.0	1.861
d384-b1-e1953-k10	0.750	10	CPU	CPU	0.0	1.303
d384-b1-e2604-k64	1.000	64	CPU	CPU	0.0	1.150
d384-b1-e3255-k10	1.250	10	CPU	GPU	21.2	1.131
d384-b1-e3906-k1	1.500	1	CPU	GPU	0.9	0.923
d768-b16-e41-k1	0.504	1	CPU	CPU	0.0	0.412
d768-b16-e61-k10	0.750	10	GPU	CPU	3.2	0.840
d768-b16-e81-k64	0.995	64	CPU	CPU	0.0	0.789
d768-b16-e102-k10	1.253	10	CPU	GPU	106.8	4.595
d768-b16-e122-k1	1.499	1	CPU	GPU	162.0	5.393
all-r1000-d128-b8-k10	1.024	10	CPU	GPU	164.5	2.351

Table 17: H1/P0 crossover cells used for the equal- E contradiction. Both cohorts use 100K×384, $B = 1, k = 10$, five runs and 5,000 calls/run/backend; source, host CPU, GPU, OS/API, and driver differ.

Environment	E	CPU P95	WGPU P95	Winner
RTX 4050 / DX12	2,000	0.5198	0.6340	CPU
RTX 4050 / DX12	3,000	0.9563	0.8144	GPU
RTX 4050 / DX12	4,000	1.3305	1.0737	GPU
RTX 4050 / DX12	6,000	1.9735	1.6033	GPU
RTX 4050 / DX12	8,000	2.5003	2.1508	GPU
RTX 4050 / DX12	10,000	2.8104	2.6041	GPU
RTX 4090 / Vulkan	1,000	0.1019	0.1777	CPU
RTX 4090 / Vulkan	2,000	0.1947	0.2630	CPU
RTX 4090 / Vulkan	3,000	0.2884	0.3673	CPU
RTX 4090 / Vulkan	4,000	0.4011	0.4143	CPU
RTX 4090 / Vulkan	6,000	1.3114	0.5916	GPU
RTX 4090 / Vulkan	8,000	2.4046	0.5934	GPU
RTX 4090 / Vulkan	10,000	3.7544	0.6807	GPU
RTX 4090 / Vulkan	100,000	5.6692	1.7747	GPU

P2’s complete reduction contains the six matched Qenlo rows summarized in Table 5, a routed dense reference verification at 14.474 ms, and FAISS CPU Flat at 11.154 ms. The latter two are time-separated diagnostics and do not turn run drift into an optimization effect. Full rows, source roles, recall, and archive identity are in `research/data/processed/archive-reanalysis/phase2_cpu_optimization.csv`.

A.7 E0/E2 partial campaign

Table 18: E0/E2 completed block summaries per engine and cell (completed/planned). $f = E/100,000$. The archive has no completion marker. The CPU $B = 16$, $E = 30K$ block 1 destination exists without a summary and counts as missing. No value is imputed.

Exp.	B	E	f	CPU	Rows	Predicate	Mask	Total
E0	1	3,000	0.03	10/10	10/10	–	–	20/20
E2	1	100	0.001	5/5	5/5	5/5	5/5	20/20
E2	1	1,000	0.01	5/5	5/5	5/5	5/5	20/20
E2	1	3,000	0.03	5/5	5/5	5/5	5/5	20/20
E2	1	10,000	0.1	5/5	5/5	5/5	5/5	20/20
E2	1	30,000	0.3	5/5	5/5	5/5	5/5	20/20
E2	1	100,000	1	5/5	5/5	5/5	5/5	20/20
E2	16	100	0.001	5/5	5/5	5/5	5/5	20/20
E2	16	1,000	0.01	5/5	5/5	5/5	5/5	20/20
E2	16	3,000	0.03	5/5	5/5	5/5	5/5	20/20
E2	16	10,000	0.1	5/5	5/5	5/5	5/5	20/20
E2	16	30,000	0.3	1/5	2/5	2/5	2/5	7/20
E2	16	100,000	1	0/5	0/5	0/5	0/5	0/20

Table 19: E2 median block P50 batch-call latency (ms) with 95% block-bootstrap intervals; five blocks unless n is shown. An $n = 1$ cell has no interval. “missing” cells were planned and not measured.

B	f	CPU	Rows	Predicate	Mask
1	0.001	0.017 [0.017, 0.018]	0.185 [0.182, 0.186]	0.998 [0.971, 1.007]	1.021 [0.985, 1.031]
1	0.01	0.111 [0.110, 0.115]	0.288 [0.285, 0.350]	1.049 [1.034, 1.063]	1.056 [1.051, 1.066]
1	0.03	0.364 [0.346, 0.419]	0.445 [0.427, 0.719]	1.126 [1.079, 1.194]	1.214 [1.046, 1.224]
1	0.1	4.190 [4.153, 4.232]	1.449 [1.088, 1.566]	1.680 [1.667, 1.706]	1.736 [1.668, 1.762]
1	0.3	9.734 [9.718, 9.856]	4.360 [3.481, 4.532]	4.344 [4.258, 4.362]	3.608 [2.767, 4.446]
1	1	11.541 [11.498, 11.596]	2.545 [2.024, 2.574]	1.535 [1.519, 1.548]	1.688 [1.681, 1.718]
16	0.001	0.278 [0.275, 0.285]	0.198 [0.196, 0.201]	1.209 [1.205, 1.367]	1.223 [1.186, 1.316]
16	0.01	1.752 [1.728, 1.795]	0.252 [0.248, 0.256]	1.459 [1.414, 1.460]	1.458 [1.353, 1.484]
16	0.03	5.692 [5.242, 5.879]	0.965 [0.806, 1.079]	1.630 [1.363, 1.650]	1.628 [1.623, 1.660]
16	0.1	62.286 [62.189, 63.411]	2.127 [2.076, 2.411]	2.263 [2.239, 2.267]	2.268 [2.262, 2.273]
16	0.3	154.648 (n=1)	5.261 [5.184, 5.337] (n=2)	5.184 [5.171, 5.197] (n=2)	4.923 [4.751, 5.095] (n=2)
16	1	missing	missing	missing	missing

Table 20: E2 median block P95 batch-call latency (ms) with 95% block-bootstrap intervals, as in Table 19. At $B = 1$, $f = 1$, all four engines report recall@10 of exactly 0.99998; all other cells report 1.

B	f	CPU	Rows	Predicate	Mask
1	0.001	0.020 [0.018, 0.020]	0.210 [0.202, 0.211]	1.054 [1.042, 1.085]	1.053 [1.036, 1.074]
1	0.01	0.120 [0.116, 0.160]	0.347 [0.327, 0.376]	1.109 [1.067, 1.134]	1.116 [1.094, 1.228]
1	0.03	0.466 [0.375, 0.615]	0.512 [0.467, 0.751]	1.264 [1.186, 1.312]	1.264 [1.195, 1.335]
1	0.1	4.718 [4.541, 5.179]	1.780 [1.195, 1.901]	2.017 [1.831, 2.067]	1.880 [1.844, 2.304]
1	0.3	10.947 [10.344, 11.150]	4.883 [4.634, 6.011]	4.696 [4.559, 5.336]	4.871 [4.589, 5.090]
1	1	12.281 [11.895, 12.439]	2.704 [2.581, 2.929]	1.651 [1.592, 1.676]	1.887 [1.778, 1.946]
16	0.001	0.363 [0.343, 0.380]	0.234 [0.207, 0.250]	1.326 [1.231, 1.397]	1.403 [1.222, 1.525]
16	0.01	1.842 [1.771, 2.637]	0.268 [0.261, 0.316]	1.502 [1.490, 1.582]	1.606 [1.477, 1.642]
16	0.03	5.874 [5.390, 6.779]	1.104 [0.961, 1.229]	1.659 [1.473, 1.712]	1.707 [1.655, 1.935]
16	0.1	74.766 [66.611, 86.755]	2.233 [2.215, 3.638]	2.345 [2.310, 2.387]	2.317 [2.294, 2.373]
16	0.3	172.614 (n=1)	5.770 [5.434, 6.106] (n=2)	5.769 [5.563, 5.975] (n=2)	5.367 [5.022, 5.712] (n=2)
16	1	missing	missing	missing	missing

Table 21: E2 paired ratios against the dense mask, notation as in Table 7. Omitting the exit-code-101 mask block at $B = 1$, $f = 0.001$ changes the rows/mask P95 ratio from 0.197 to 0.198.

B	f	n	Rows/mask P50	Rows/mask P95	Predicate/mask P50	Predicate/mask P95
1	0.001	5	0.181 [0.178, 0.189] [†]	0.197 [0.190, 0.203] [†]	0.986 [0.941, 1.006]	0.996 [0.981, 1.024]
1	0.01	5	0.273 [0.269, 0.332] [†]	0.301 [0.286, 0.337] [†]	0.995 [0.983, 1.005]	0.975 [0.909, 1.017]
1	0.03	5	0.408 [0.362, 0.661]	0.429 [0.369, 0.625]	0.983 [0.919, 1.031]	1.001 [0.947, 1.033]
1	0.1	5	0.869 [0.626, 0.889]	0.877 [0.635, 1.011]	0.963 [0.960, 1.023]	0.993 [0.839, 1.099]
1	0.3	5	1.179 [1.012, 1.330]	1.009 [0.910, 1.234]	1.180 [0.977, 1.554]	1.005 [0.907, 1.096]
1	1	5	1.514 [1.178, 1.515]	1.422 [1.411, 1.640]	0.909 [0.895, 0.911]	0.877 [0.843, 0.913]
16	0.001	5	0.164 [0.153, 0.167] [†]	0.176 [0.145, 0.189] [†]	1.012 [0.918, 1.068]	0.976 [0.869, 1.081]
16	0.01	5	0.173 [0.170, 0.186] [†]	0.181 [0.160, 0.197] [†]	0.993 [0.968, 1.079]	0.935 [0.908, 1.071]
16	0.03	5	0.581 [0.496, 0.651]	0.652 [0.497, 0.697]	0.996 [0.837, 1.006]	0.979 [0.836, 1.003]
16	0.1	5	0.936 [0.918, 1.063]	0.973 [0.941, 1.570]	0.999 [0.987, 1.002]	1.012 [0.994, 1.023]
16	0.3	2	1.069 [1.017, 1.123]	1.075 [1.069, 1.082]	1.054 [1.020, 1.088]	1.076 [0.974, 1.190]
16	1	0	missing	missing	missing	missing

Per-block values, including within-block run spread, process return code, and start time, are in `research/data/processed/runpod-e0-e2-analysis-20260924/block_metrics.csv`. The E0 modified- z rule flags CPU block 6 at P95 (0.756 ms) and gpu-rows blocks 5 and 6 at P50 (0.432 and 0.366 ms) and P95 (0.506 and 0.424 ms); none is removed. Two derived files report different bootstrap lower endpoints for the E0 gpu-rows P95 median, 0.614 ms in `e0_noise.csv` and 0.625 ms in `latency_summary.csv`, because they use different seeds; Table 6 uses the former. Minimum per-call recall@10 across 2,181,693 sample rows is 0.9, while every summary passes its aggregate target. Recorded GPU telemetry spans 210–2,775 MHz SM clock, 23–35 °C, and 12.24–131.26 W, as before/after snapshots only. Driver, CUDA, and host-CPU identities are not recorded in the archive.

A.8 Historical A6000 distributions

Table 22: H4 strict real TopK, $1M \times 768$, $E = 10,026$, $B = 1$, $k = 10$, 5,000 calls/system. Shared prefiltered matrix, host completed calls. cuVS fails correctness and is excluded from qualified comparisons. CUDA prototype is unshipped research code.

System	P50	P95	P99	FP64 recall
cuVS (disqualified)	0.3615	0.3846	0.4131	0.90000
FAISS GPU Flat	0.1261	0.1322	0.1458	1.00000
NumPy CPU exhaustive	0.1217	0.1973	0.2283	1.00000
CUDA prototype	0.1600	0.1707	0.1809	1.00000

Table 23: H5 generated normalized FP32, $1M \times 768$, $B = 1$, $k = 10$, 500 calls/system/cell across five runs. Completed-call distribution percentiles; unordered top-10 agreement, not independent FP64 truth.

Eligible	System	P50	P95	P99
1000	FAISS GPU Flat	0.0672	0.0857	0.0952
1000	CUDA prototype	0.0775	0.0975	0.1155
10000	FAISS GPU Flat	0.1272	0.1373	0.1505
10000	CUDA prototype	0.1438	0.1557	0.1741
100000	FAISS GPU Flat	0.7755	0.7887	0.8001
100000	CUDA prototype	0.5559	0.5673	0.5763
1000000	FAISS GPU Flat	6.2736	6.3475	6.4357
1000000	CUDA prototype	4.6508	4.6683	4.7364

A.9 Historical Linux libraries

Descriptive Linux library cohort: same data and host, 1,500 calls/system

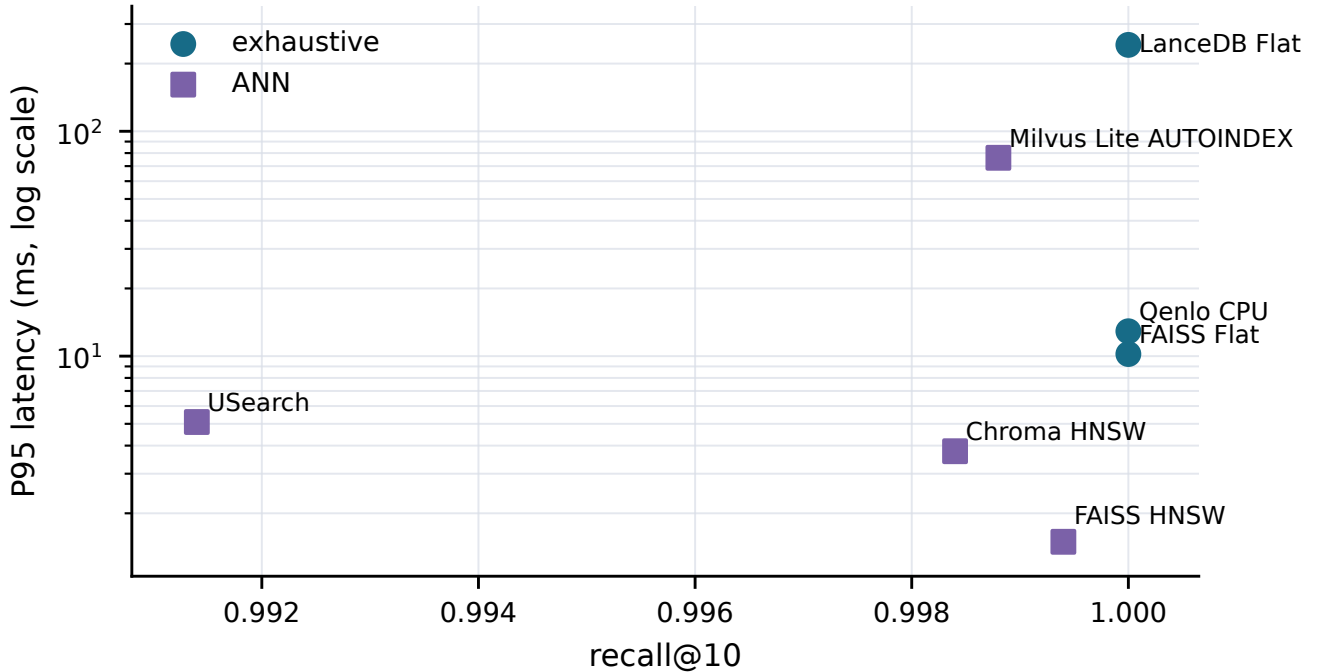


Figure 14: H3 bb51987, Linux CPU library context, separately prepared AG News $100K \times 384$, unfiltered $B = 1$, $k = 10$. Three runs/1,500 calls per system; completed library-call P95. Recall accompanies latency; algorithms, persistence, and APIs differ. This figure is descriptive within the host, not a product leaderboard.

Table 24: H3 complete retained library context. ANN recall is part of each operating point; exhaustive engines use different APIs. Native GPU failed Vulkan initialization and has no timed row.

System	Search	P95	FP64 recall
Qenlo CPU	exhaustive	12.9014	1.00000
USearch	ANN	5.0933	0.99140
FAISS Flat	exhaustive	10.2171	1.00000
FAISS HNSW	ANN	1.4939	0.99940
Chroma HNSW	ANN	3.7767	0.99840
Milvus Lite AUTOINDEX	ANN	76.2294	0.99880
LanceDB Flat	exhaustive	242.2557	1.00000

A.10 Android aggregate records

Table 25: H6 all 21 supplied Android cells. $D = 384, k = 10$; B is batch size and n is the reported sample count. All recall@10 values are 1.0, and all fallback counts are zero. Automatic rows use selective predicates and are not comparable with dense rows. No raw-call reconstruction is possible.

Suite	Cell	Rows	B	n	P50	P95	P99
quick	cpu-exact-all	10000	1	16	1.8410	5.7670	5.7670
quick	gpu-exact-all	10000	1	16	8.7070	12.4870	12.4870
quick	gpu-native-batch-8	10000	8	16	5.8860	5.8860	5.8860
quick	gpu-ivf-flat-recall-95	10000	1	16	3.6160	8.0560	8.0560
quick	gpu-ivf-sq8-recall-95	10000	1	16	4.7460	7.3160	7.3160
quick	automatic-selective-cpu-route	10000	1	16	0.0230	0.0250	0.0250
quick	automatic-selective-batch-8	10000	8	16	2.3100	2.3100	2.3100
full	cpu-exact-all	100000	1	64	18.3080	29.4360	41.5590
full	gpu-exact-all	100000	1	64	13.8060	16.9750	48.3750
full	gpu-native-batch-8	100000	8	64	10.6080	11.1010	11.1010
full	gpu-ivf-flat-recall-95	100000	1	64	8.8510	21.6170	32.4310
full	gpu-ivf-sq8-recall-95	100000	1	64	20.3940	26.5780	29.1260
full	automatic-selective-cpu-route	100000	1	64	0.2560	0.3110	0.3230
full	automatic-selective-batch-8	100000	8	64	10.0390	15.5710	15.5710
soak	cpu-exact-all	100000	1	512	25.7990	27.0190	27.5720
soak	gpu-exact-all	100000	1	512	14.2740	17.6780	25.7690
soak	gpu-native-batch-8	100000	8	512	10.2900	11.1310	12.3880
soak	gpu-ivf-flat-recall-95	100000	1	512	13.9770	21.4530	24.5490
soak	gpu-ivf-sq8-recall-95	100000	1	512	27.3590	37.0900	40.5810
soak	automatic-selective-cpu-route	100000	1	512	0.3570	0.5440	0.7430
soak	automatic-selective-batch-8	100000	8	512	8.1740	9.8900	12.4410

A.11 Intel Arc aggregate records

Table 26: H7 all 21 Intel Arc/Vulkan cells, app 0.1.0, $D = 384$, $k = 10$. All reported recall is 1.0, with no fallback. The archive contains two quick reports and one soak; labels follow the embedded JSON. Automatic cells are selective and separate from dense rows. Raw series are unavailable.

Suite	Cell	Rows	B	n	P50	P95	P99
quick1	CPU exhaustive	10000	1	16	0.971	1.219	1.219
quick1	WGPU exhaustive	10000	1	16	1.692	2.630	2.630
quick1	WGPU batch 8	10000	8	16	0.196	0.196	0.196
quick1	IVF-Flat	10000	1	16	0.470	0.905	0.905
quick1	IVF-SQ8	10000	1	16	1.707	1.997	1.997
quick1	Auto selective CPU	10000	1	16	0.017	0.021	0.021
quick1	Auto selective WGPU	10000	8	16	0.325	0.325	0.325
quick2	CPU exhaustive	10000	1	16	0.864	1.787	1.787
quick2	WGPU exhaustive	10000	1	16	0.949	1.589	1.589
quick2	WGPU batch 8	10000	8	16	0.183	0.183	0.183
quick2	IVF-Flat	10000	1	16	0.387	0.540	0.540
quick2	IVF-SQ8	10000	1	16	1.559	3.554	3.554
quick2	Auto selective CPU	10000	1	16	0.015	0.018	0.018
quick2	Auto selective WGPU	10000	8	16	0.390	0.390	0.390
soak	CPU exhaustive	100000	1	512	14.639	16.486	19.838
soak	WGPU exhaustive	100000	1	512	3.707	4.444	4.724
soak	WGPU batch 8	100000	8	512	1.100	1.245	1.293
soak	IVF-Flat	100000	1	512	2.018	2.704	3.172
soak	IVF-SQ8	100000	1	512	16.401	22.797	25.426
soak	Auto selective CPU	100000	1	512	0.191	0.273	0.343
soak	Auto selective WGPU	100000	8	512	0.495	0.555	0.866

B Claim ledger and provenance

The definitive machine-readable ledger is `paper/tables/claim-to-artifact.json`, with a CSV index at the same stem. It incorporates audited historical, campaign, and system claims together with every matrix row. Each measurement records its source, revision or hash, environment, data provenance, workload, filter, timing, sample or repetition unit, oracle, memory scope, qualification, and limitations. Unknown provenance is explicit rather than inferred from a nearby cohort. Full machine paths remain outside the narrow printed tables for legibility.

Table 27: Headline claim families and authoritative artifacts. Paths are relative to the repository; complete hashes and per-row fields are in the ledger.

Claim	Result / qualification	Artifact family
H1/H1-regret	2K–3K reversal; up to 23.9% counterfactual regret, one historical laptop	research/data/raw/2026-09-02-native-crossover/ ; per-cell samples/runs/configurations
H0	Historical representation/endpoints; different revisions explicitly separated	benchmarks/2026-08-28/real/
H2	One-pass/cache preparation; five sequential runs, no CPU recalibration	research/artifacts/qenlo-phase1-eligibility-2026-09-04.tar.gz
S2	Twelve corrected 768D rows, recall 1.0; candidate selector	Campaign deep768/rtx4090-eu-ro-secure-deep768-admission-fix/
S1-selector	Five wins/seven losses; rejected unconditional optimization	Matrix paired baseline-gpu/current-gpu ; source archives
S1-real/ANN	Real fixture; seven qualified Chroma pairs; failed gates and USearch visible	Campaign deep/ and reference/ ; matrix and raw samples
S1-lifecycle	Search-only P95; separate mutation/opening; zero/final-one rebuilds	Reference lifecycle-current-gpu samples/summary
S2-resources	Device allocations, process RSS, transfers, nested phases	Corrected native samples, process time logs, matrix

Table 28: Historical, platform, accounting, and semantic claim families (continued).

Claim	Result / qualification	Artifact family
H4/H5	Real strict negative; synthetic formulation reversal	benchmark-results/2026-09-02-runpod-a6000-strict-research-gate/ ; research/data/raw/2026-09-02-a6000-cuda/heldout/
H3	Seven CPU library rows, no WGPU timing	benchmark-results/2026-09-02-runpod-archive/retained-archive/artifacts/
H6/H7	Physical Android and Arc, aggregate-only	research/data/raw/2026-09-02-android/ ; benchmarks/2026-08-31/device-lab/intel-arc/reports.json
R1	Frozen <i>EDB</i> rule: 8/16 wrong, 235.7% max regret, rejected	research/data/raw/alpha5-router-heldout-rtx4050.tar.gz ; archive reanalysis
P0	Separate 4K–6K reversal; equal- <i>E</i> contradiction with H1	research/artifacts/qenlo-phase0-runpod-complete-2026-09-04.tar.gz
P2	Sparse CPU gains, dense regression, FAISS CPU context	research/artifacts/qenlo-phase2-a40-2026-09-04-final.tar.gz
E0	Same-cell block ratios near 2×; GPU bimodality associated with clock snapshots	research/data/raw/runpod-e0-e2-partial-20260924.tar.gz ; research/data/processed/runpod-e0-e2-analysis-20260924/e0_noise.csv
E2	227/260 summaries; sparse rows 3.2–6.1× faster than predicate/mask; no $f^*(16)$	Same archive; paired_comparisons.csv , latency_summary.csv , coverage.csv
E0/E2 witness	Same-host <i>EDB</i> ordering contradiction, post hoc	research/data/processed/runpod-e0-e2-mechanisms-20260924/edb_ordering_witness.csv
Harness	Batch transfer counters overstated <i>B</i> -fold; S2 batch-8 corrected	crates/qenlo-bench/src/main.rs ; mechanisms_diagnostics_by_cell.csv
Accounting	Account daily spend, zero retained campaign resources	Campaign ledger.jsonl , billing captures , campaign-summary.json
Semantics	Current source contracts; not retrospective performance	paper/audit/system-ledger.json

B.1 Source and result archives

The corrected replay records Python 3.12.3, FAISS 1.15.0, NumPy 2.3.3, and PyTorch 2.9.1+cu128. Driver and runtime version labels describe different layers: the NVIDIA driver reports CUDA 13.0, while the PyTorch build targets CUDA 12.8. Per-configuration environment captures remain authoritative for other hosts.

The campaign root is `research/artifacts/runpod-small-2026-09-05/`. Full identities are:

S0 frozen source. `f37e744c7ee4054a74c4b4181f2dde61dade4d677bbc5493b9a36b61d165a45d`

S1 experimental source. `730c050065e6786972a7be4f9bcb619168becdc943ce1122d1583c61f66409e3`

S2 corrected source. `bb195fedb6519c26598c6c103e7e182982c2eddd0a06d9c2cf54eb5cb5ceeb19`

S2 result archive. `9e322d615c0bbcd44616faa831493ca030dd49bbce7fe0c4386f49e3b8532f0c`

H2 eligibility archive. `26e0ccc057c59c0fb4687f8d85a923e88b7bb25257e4e551e22cffe13c1b821f`

R1 held-out router archive. `8e3b5fe6ec6ab9f295752ec894c89b094c4ddf4dd0ee38849d9d78c9a7fcc01e`

P0 complete archive. `7292fd1cd0e7669c9febeaa2d49baedee747811ed65f6bc704ecbcb6e61f51ea`

P2 CPU archive. `ac3afda446f4943eab128cff458402f7559be53b96c3cf8ce44fcbbd7f8be4df`

E0/E2 partial archive. `e4ca1336757cf55a5f5150a8bcc601ab3538cb7e9e9c2270c137e5a72fd51ed0`

E0/E2 attested source. `e5d85adbb638caa05a99fb8d1ef0a96147091cf0`

E0/E2 attested binary. `b28e16ffda779304f10bc6fea61ec6fe368ce0923590593e2ab27443bc70285b`

E0/E2 attested dataset. `ba87a322b6846ce225ce54140cf00ac33250271447ef8f9f83df246131719cba`

E0/E2 runner (recomputed). `c0ae079bc4ddbac8ad4efef2a4a25c5be8418e9a1276edb5bcd7dda71a24ae56`

S2 result integrity is distinct from S2 source identity. The corrected result manifest has 74 non-self entries. The full campaign verifier checks eight retained result archives and nine checksum manifests, covering 1,053 entries and skipping four self-manifest entries. It also checks source roles and the identity of the local and baseline WGSL. Checksums establish byte integrity, not scientific qualification.

B.2 Discrepancies resolved during authoring

The audit reconciled disagreements instead of choosing flattering prose. It records that (i) “current” source archives still contain the rejected selector; (ii) the matrix lifecycle scope string is broader than the separate raw timers; (iii) external replay preparation omits eligibility traversal despite its field name; (iv) completion is not identical to qualification; (v) historical endpoint revisions differ; (vi) Intel Arc’s embedded quick label overrides an external full label; and (vii) final spend is account and daily accounting. Bibliographic corrections include the DTU dataset’s publication year, the SPANN title, DiskANN author spelling, and the full Filtered-DiskANN author list. Audit notes retain these resolutions.

B.3 Excluded evidence and unresolved claims

No raw evidence or old figure was deleted. The historical inventory hashes every top-level figure file, and regenerated figures live in a separate `paper/figures/final/` directory. Older dimension, matched-size, metadata-correlation, and heatmap graphics do not establish unmeasured experiments. Redundant crossover and selectivity plots are superseded by explicitly scoped panels. The old architecture figure is retained, while the paper explains canonical state in equations and prose.

An incomplete historical CPU development run and duplicate cloud mirrors are excluded from the counts. H3 native Vulkan initialization failed, so there is no GPU latency to rank. Historical Qdrant was deliberately outside the retained comparison because its local-mode search boundary differed; no conclusion about its performance follows. Provider-compatible TopK experiments used an inclusive predicate that yielded 10,148 eligible rows and supplied truth, whereas strict H4 used 10,026 rows and an independent oracle. These experiments remain exploratory negative context and are not pooled with H4. Their approximately one-percent prototype did not

beat FAISS, and other provider cells had correctness disagreements. cuVS numerical and runtime failures remain failures, not qualified exact results.

The archive does not test a complete multidimensional routing surface, controlled fixed- E with varying- N , controlled metadata and vector correlation, a tuned filtered-ANN frontier, a successful calibrated router on held-out data, concurrency, energy, total VRAM peaks, long-running mutation churn, arbitrary crash recovery, MPS, or physical iOS. R1 supplies a held-out rejection, not a positive adaptive result. Historical mobile results cannot certify the final revision. The S2 rerun with the final local selector is absent. E2 lacks its batch-16 cells at $E = 100K$ and most of $E = 30K$, an automatic-path arm, continuous GPU telemetry, and repeated noise controls outside one sparse batch-one cell. These gaps constrain the claims rather than licensing extrapolation.

B.4 Reproduction and audit procedure

The archive-wide entry point is `research/scripts/analyze_full_archive.py`; it verifies R1/P0/H2/P2 hashes, parses archives without extracting over retained data, emits the routing/CPU reductions, and writes a disposition ledger for tracked or selected archive-internal sample series. Other paper-only entry points are `paper/scripts/reduce_final_evidence.py`, `generate_final_tables.py`, `generate_final_figures.py`, and `verify_campaign_claims.py`. Historical raw checks are in `paper/audit/verify_historical_raw.py`. Reductions go to `paper/audit/reduced/` and `research/data/processed/archive-reanalysis/`. Android values are regenerated from retained aggregate records rather than unavailable raw series. Numerical tables and figure inputs are traceable through `paper/audit/figure-sources.json`. A clean LaTeX build, citation, reference, and box checks, extracted text, and page renders are recorded in the paper verification output. E0/E2 has three read-only stages: `research/data/processed/runpod-e0-e2-analysis-20260924/analyze.py` and `render_report.py` produce the latency statistics from a Python-extracted copy of the archive; `research/scripts/audit_e0_e2_mechanisms.py` reads the tarball in place and produces diagnostics, clock associations, and the ordering witness; and `paper/scripts/generate_e0e2_tables.py` writes the table bodies. Cargo tests separately verify the post-audit batch-counter, range-materialization, and required-predicate changes; they do not regenerate or revise campaign measurements. No command in this authoring workflow creates cloud resources or runs a performance benchmark.